L2: Data scraping and database management with Python

Course Code: COMP7415

Lecturer: Tony Lam

Agenda

- Understand the basic website structure
- Web scraping using different python libraries
- The limitations of web scraping
- Create a simple database using python
- Understand the concept of adjusted price
- Database design for financial market data storage

Installation of Python

- Python: https://www.python.org/downloads/
- Anaconda (suggested): https://www.anaconda.com/download/success
  - easier management for different environments
  - pre-installed data analysis packages (eg. numpy, pandas, scipy, etc)

What's Data/Web Scraping?

- Automated process to extract data from websites
- Common use cases in algo-trading:
  - Real-time data collection (eg. economic indicators, stock prices, etc)
  - Market sentiment analysis (eg. news, forum and blogs, social media)
  - Corporate analysis (eg. product announcements, annual reports, director changes, etc)
  - Alternative data (eg. weather, web traffic, etc)

Basic Web structure

A website is usually made of

1. HTML (HyperText Markup Language)
2. CSS (Cascading Style Sheets)
3. JavaScript

HTML

- Purpose: Defines the structure and content of a webpage.
- Elements: Uses tags to create elements such as headings, paragraphs, links, images, tables, etc.
- Markup Language: Not a programming language; it’s a markup language used to structure content.

Common HTML Tags

Tag	Description
<html>	Defines the root of an HTML document.
<head>	Contains meta-information about the HTML document, such as title, styles, and scripts.
<title>	Sets the title of the HTML document, displayed in the browser's title bar or tab.
<body>	Contains the content of the HTML document, including text, images, links, and other elements.
<h1>, <h2>, <h3>, <h4>, <h5>, <h6>	Defines headings of different sizes, with <h1> being the largest and <h6> the smallest.
<p>	Defines a paragraph of text.
<a>	Creates a hyperlink, linking to another webpage, file, or location within the same page.
<img>	Inserts an image into the HTML document.
	Defines a division or container for grouping HTML elements and applying CSS styles.
	Defines an inline container for styling a specific portion of text or content.
,	Creates an unordered or ordered list, respectively, containing list items ( ).
	Defines a list item within an unordered or ordered list.
<table>	Creates a table for organizing data into rows and columns.
<tr>	Defines a table row within a <table> element.
<td>	Defines a table cell within a <tr> element.
<th>	Defines a header cell within a <tr> element in a table.
<form>	Creates an HTML form for collecting user input.
<input>	Defines an input control element within a form, such as text fields, checkboxes, or buttons.
<label>	Associates a label with an input element, improving accessibility and usability.
<button>	Defines a clickable button within a form or webpage.
<textarea>	Creates a multiline text input control within a form.
<iframe>	Embeds another HTML page or external content within the current document.
<script>	Embeds or links to client-side scripts, such as JavaScript.
<style>	Contains CSS rules for styling HTML elements within the document.

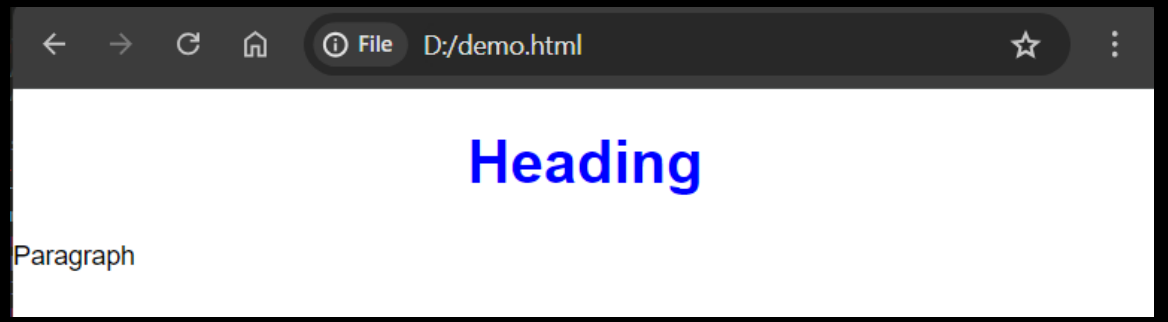
HTML Example

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Page Title</title>
</head>
<body>
    <h1>Heading</h1>
    <p>Paragraph</p>
</body>
</html>

```

CSS

- Purpose: Controls the visual presentation and layout of a webpage
- Styling: Defines styles for HTML elements, such as colors, fonts, spacing, and positioning.
- Cascading Rules: Multiple styles can be applied to an element, with rules for which styles take precedence.

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Page Title</title>

    <style type="text/css">
    body {
      font-family: Arial, sans-serif;
      margin: 0;
    }

    h1 {
      color: blue;
      text-align: center;
    }

    p {
      font-size: 14px;
      line-height: 1.5;
    }
    </style>

</head>
<body>
    <h1>Heading</h1>
    <p>Paragraph</p>
</body>
</html>
```

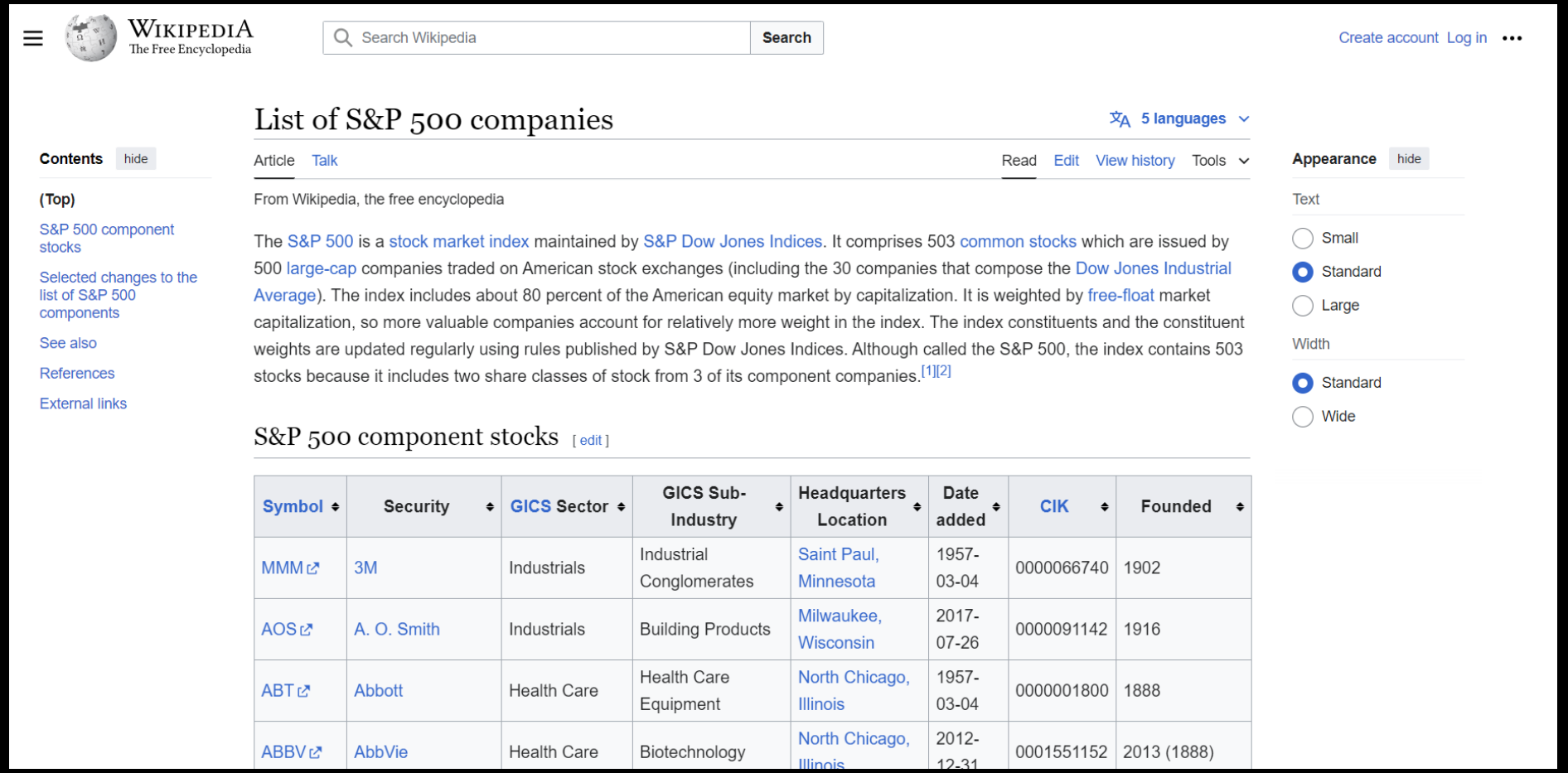


JavaScript

- Purpose: Adds interactivity and dynamic behavior to webpages.
- Programming Language: A scripting language that runs in the browser (client-side) and/or on the server (server-side with Node.js).
- Manipulation: Can manipulate HTML and CSS to change content and styles dynamically.
- Event Handling: Responds to user actions like clicks, form submissions, and keyboard input.

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Page Title</title>

    <style type="text/css">
    body {
      font-family: Arial, sans-serif;
      margin: 0;
    }

    h1 {
      color: blue;
      text-align: center;
    }

    p {
      font-size: 14px;
      line-height: 1.5;
    }
    </style>

    <script type="text/javascript">
    document.addEventListener("DOMContentLoaded", function() {
      document.querySelector("h1").style.color = "red";
    });
    </script>


</head>
<body>
    <h1>Heading</h1>
    <p>Paragraph</p>
</body>
</html>
```

Task: Scrape S&P 500 stock composites



WIKIPEDIA
The Free Encyclopedia

Search Wikipedia | Search

Create account | Log in

List of S&P 500 companies

5 languages

Article | Talk | Read | Edit | View history | Tools

From Wikipedia, the free encyclopedia

Contents hide

(Top)
S&P 500 component stocks
Selected changes to the list of S&P 500 components
See also
References
External links

Appearance hide

Text
Small
Standard
Large

Width
Standard
Wide

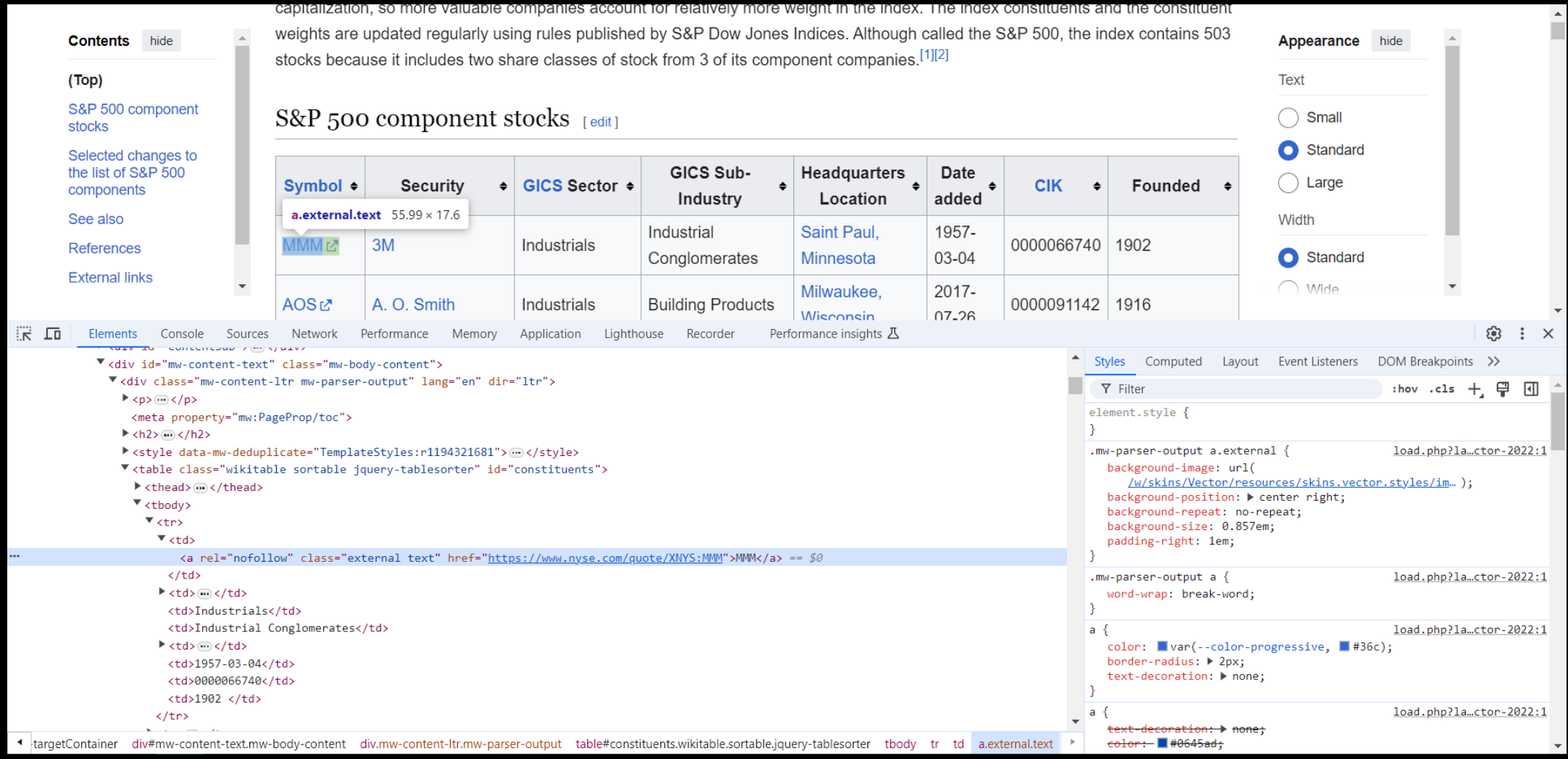
The S&P 500 is a stock market index maintained by S&P Dow Jones Indices. It comprises 503 common stocks which are issued by 500 large-cap companies traded on American stock exchanges (including the 30 companies that compose the Dow Jones Industrial Average). The index includes about 80 percent of the American equity market by capitalization. It is weighted by free-float market capitalization, so more valuable companies account for relatively more weight in the index. The index constituents and the constituent weights are updated regularly using rules published by S&P Dow Jones Indices. Although called the S&P 500, the index contains 503 stocks because it includes two share classes of stock from 3 of its component companies.[1][2]

S&P 500 component stocks [edit]

Symbol	Security	GICS Sector	GICS Sub-Industry	Headquarters Location	Date added	CIK	Founded
MMM	3M	Industrials	Industrial Conglomerates	Saint Paul, Minnesota	1957-03-04	0000066740	1902
AOS	A. O. Smith	Industrials	Building Products	Milwaukee, Wisconsin	2017-07-26	0000091142	1916
ABT	Abbott	Health Care	Health Care Equipment	North Chicago, Illinois	1957-03-04	0000001800	1888
ABBV	AbbVie	Health Care	Biotechnology	North Chicago, Illinois	2012-12-31	0001551152	2013 (1888)

https://en.wikipedia.org/wiki/List_of_S%26P_500_companies

Inspect the website structure



Web Scraping with Python

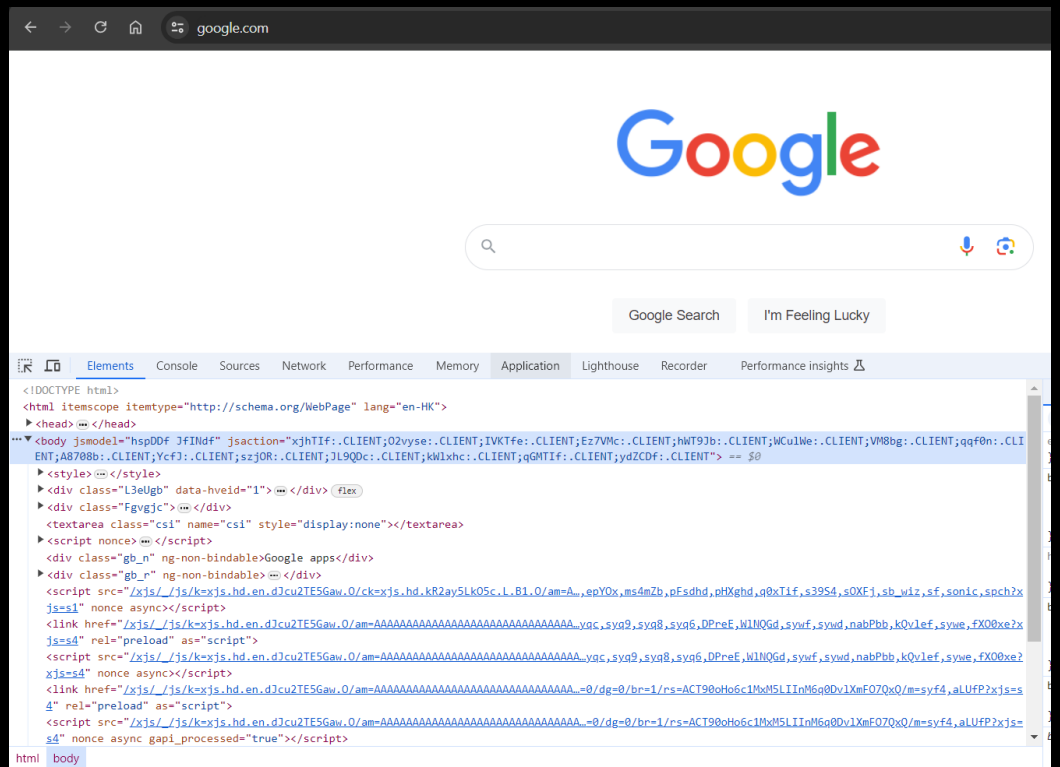
BeautifulSoup

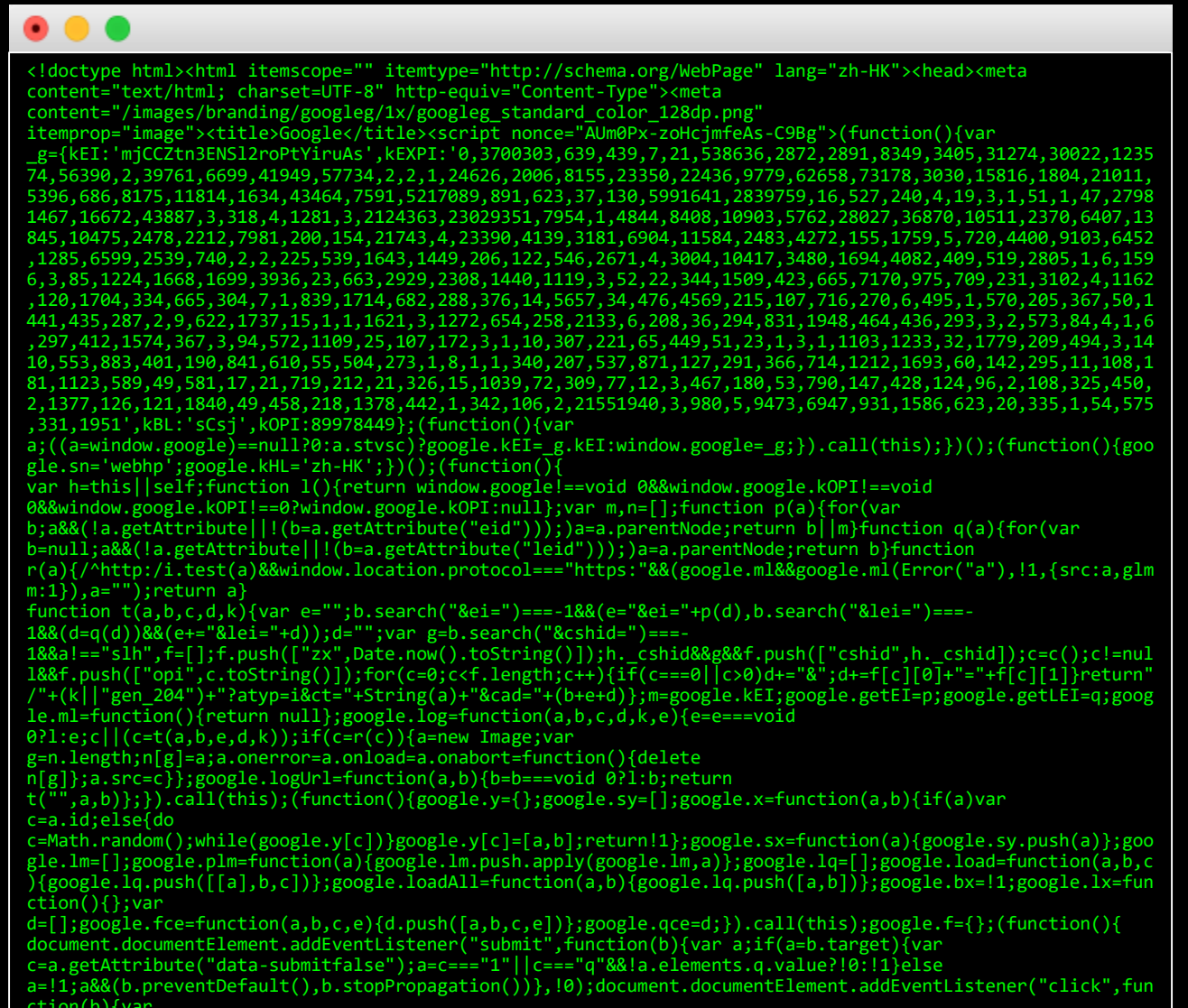
- Popular python library for searching and parsing HTML and XML data
- Official Doc: https://beautiful-soup-4.readthedocs.io/en/latest/

```
pip install beautifulsoup4
pip install requests
```

Download raw content of a web page

```
import requests

r = requests.get("https://google.com")
text = r.text

print(text)
```





Quick Example

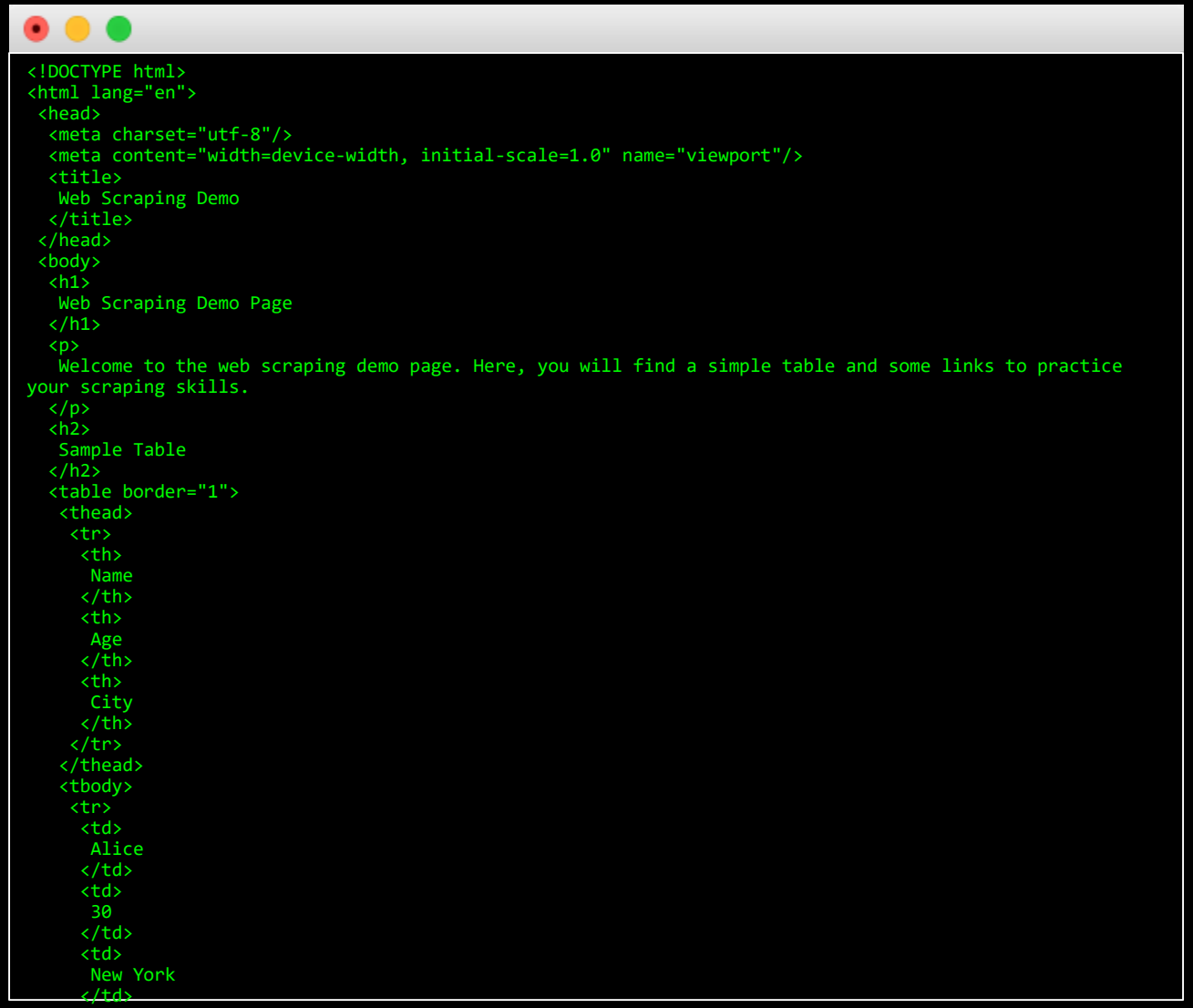
```
<!DOCTYPE html><html lang="en"><head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>Web Scraping Demo</title></head><body> <h1>Web Scraping Demo Page</h1> <p>Welcome to the web scraping demo page. Here, you will find a simple table and some links to practice your scraping skills.</p> <h2>Sample Table</h2> <table border="1"> <thead> <tr> <th>Name</th> <th>Age</th> <th>City</th> </tr> </thead> <tbody> <tr> <td>Alice</td> <td>30</td> <td>New York</td> </tr> <tr> <td>Bob</td> <td>25</td> <td>Los Angeles</td> </tr> <tr> <td>Charlie</td> <td>35</td> <td>Chicago</td> </tr> </tbody> </table> <h2>Useful Links</h2> <p>Check out the following resources for more information:</p> <ul> <li><a href="https://www.python.org/">Python Official Website</a></li> <li><a href="https://www.crummy.com/software/BeautifulSoup/">Beautiful Soup Documentation</a></li> <li><a href="https://requests.readthedocs.io/">Requests: HTTP for Humans</a></li> </ul></body></html>
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Web Scraping Demo</title>
</head>
<body>
    <h1>Web Scraping Demo Page</h1>
    <p>Welcome to the web scraping demo page. Here, you will find a simple table and some links to practice your scraping skills.</p>
    <h2>Sample Table</h2>
    <table border="1">
        <thead>
            <tr>
                <th>Name</th>
                <th>Age</th>
                <th>City</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>Alice</td>
                <td>30</td>
                <td>New York</td>
            </tr>
            <tr>
                <td>Bob</td>
                <td>25</td>
                <td>Los Angeles</td>
            </tr>
            <tr>
                <td>Charlie</td>
                <td>35</td>
                <td>Chicago</td>
            </tr>
        </tbody>
    </table>
    <h2>Useful Links</h2>
    <p>Check out the following resources for more information:</p>
    <ul>
        <li><a href="https://www.python.org/">Python Official Website</a></li>
        <li><a href="https://www.crummy.com/software/BeautifulSoup/">Beautiful Soup Documentation</a></li>
        <li><a href="https://requests.readthedocs.io/">Requests: HTTP for Humans</a></li>
    </ul>
</body>
</html>
```

Print website elements in a nice format

- prettify()

```
import requests
from bs4 import BeautifulSoup

text = '<!DOCTYPE html><html lang="en"><head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>Web Scraping Demo</title></head><body> <h1>Web Scraping Demo Page</h1> <p>Welcome to the web scraping demo page. Here, you will find a simple table and some links to practice your scraping skills.</p> <h2>Sample Table</h2> <table border="1"> <thead> <tr> <th>Name</th> <th>Age</th> <th>City</th> </tr> </thead> <tbody> <tr> <td>Alice</td> <td>30</td> <td>New York</td> </tr> <tr> <td>Bob</td> <td>25</td> <td>Los Angeles</td> </tr> <tr> <td>Charlie</td> <td>35</td> <td>Chicago</td> </tr> </tbody> </table> <h2>Useful Links</h2> <p>Check out the following resources for more information:</p> <ul> <li><a href="https://www.python.org/">Python Official Website</a></li> <li><a href="https://www.crummy.com/software/BeautifulSoup/">Beautiful Soup Documentation</a></li> <li><a href="https://requests.readthedocs.io/">Requests: HTTP for Humans</a></li> </ul></body></html>'

soup = BeautifulSoup(text, 'html.parser')

print(soup.prettify())
```

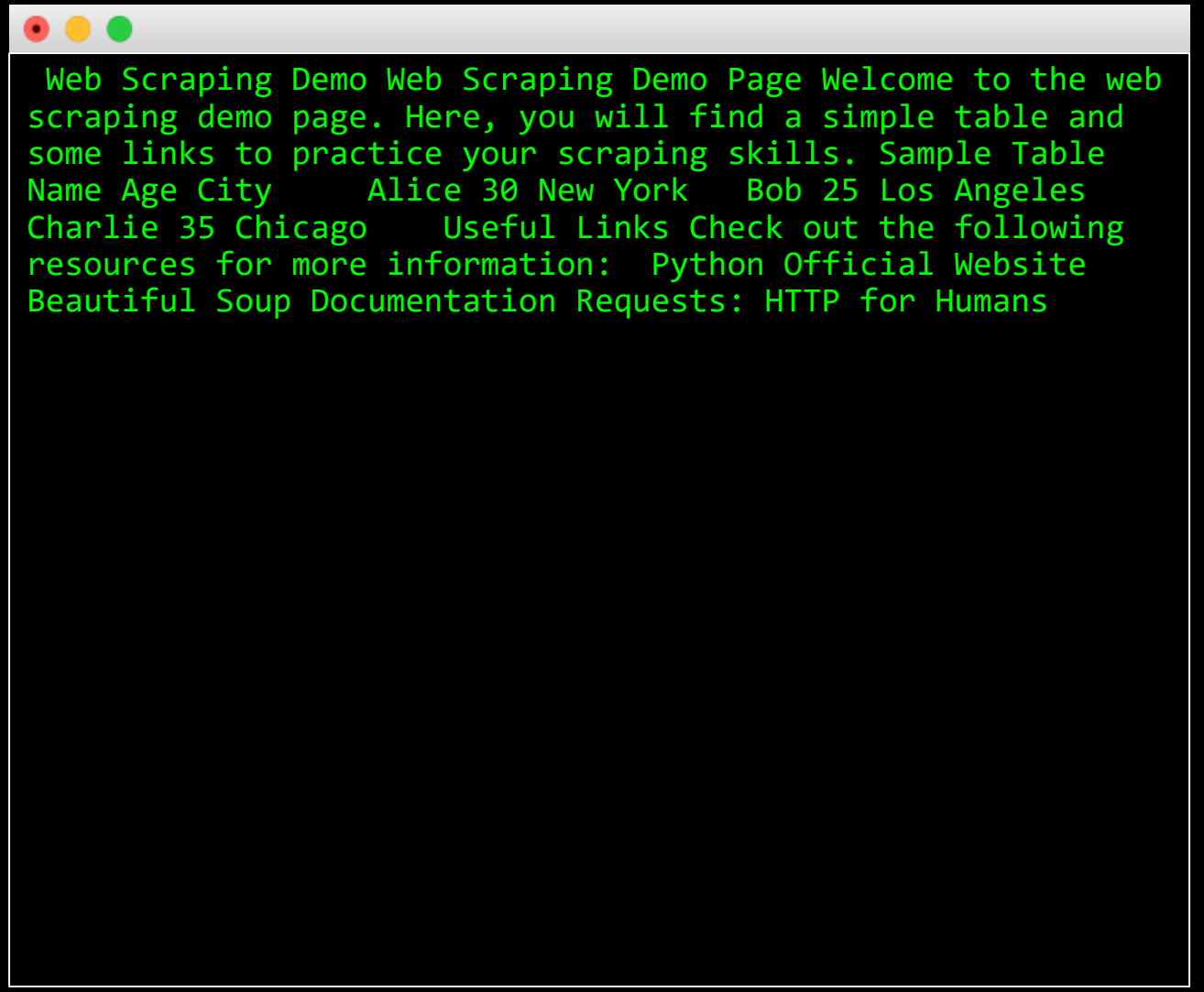


```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="utf-8"/>
  <meta content="width=device-width, initial-scale=1.0" name="viewport"/>
  <title>
   Web Scraping Demo
  </title>
 </head>
 <body>
  <h1>
   Web Scraping Demo Page
  </h1>
  <p>
   Welcome to the web scraping demo page. Here, you will find a simple table and some links to practice
your scraping skills.
  </p>
  <h2>
   Sample Table
  </h2>
  <table border="1">
   <thead>
    <tr>
     <th>
      Name
     </th>
     <th>
      Age
     </th>
     <th>
      City
     </th>
    </tr>
   </thead>
   <tbody>
    <tr>
     <td>
      Alice
     </td>
     <td>
      30
     </td>
     <td>
      New York
     </td>
```

Extract all Text in a website

- get_text()

```
import requests
from bs4 import BeautifulSoup

text = '<!DOCTYPE html><html lang="en"><head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>Web Scraping Demo</title></head><body> <h1>Web Scraping Demo Page</h1> <p>Welcome to the web scraping demo page. Here, you will find a simple table and some links to practice your scraping skills.</p> <h2>Sample Table</h2> <table border="1"> <thead> <tr> <th>Name</th> <th>Age</th> <th>City</th> </tr> </thead> <tbody> <tr> <td>Alice</td> <td>30</td> <td>New York</td> </tr> <tr> <td>Bob</td> <td>25</td> <td>Los Angeles</td> </tr> <tr> <td>Charlie</td> <td>35</td> <td>Chicago</td> </tr> </tbody> </table> <h2>Useful Links</h2> <p>Check out the following resources for more information:</p> <ul> <li><a href="https://www.python.org/">Python Official Website</a></li> <li><a href="https://www.crummy.com/software/BeautifulSoup/">Beautiful Soup Documentation</a></li> <li><a href="https://requests.readthedocs.io/">Requests: HTTP for Humans</a></li> </ul></body></html>'

soup = BeautifulSoup(text, 'html.parser')

print(soup.get_text())
```



Extract the 1st paragraph <p>

- find(“p”)

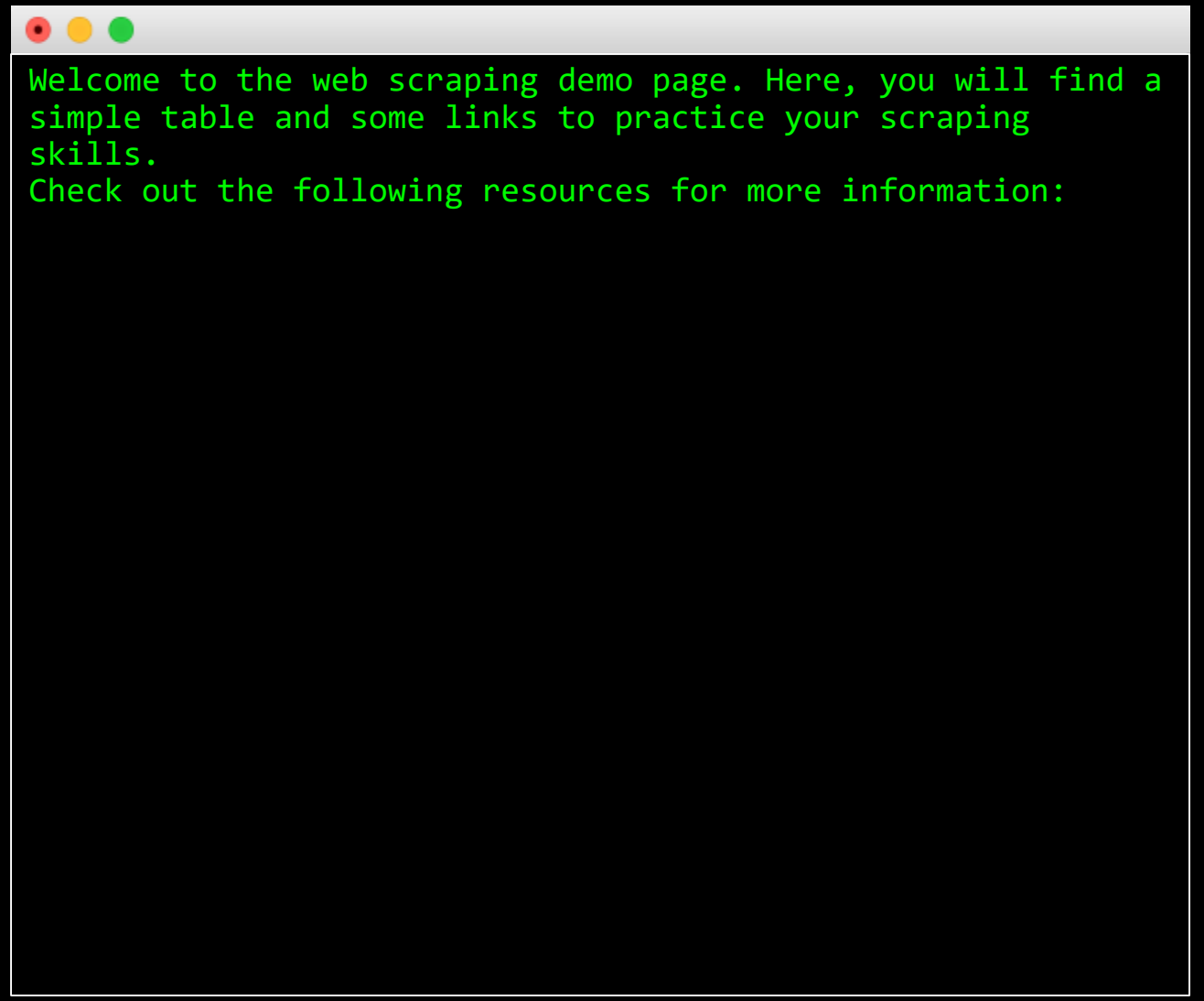
```
import requests
from bs4 import BeautifulSoup

text = '<!DOCTYPE html><html lang="en"><head> <meta charset="UTF-8"> <meta
name="viewport" content="width=device-width, initial-scale=1.0"> <title>Web Scraping
Demo</title></head><body> <h1>Web Scraping Demo Page</h1> <p>Welcome to the web
scraping demo page. Here, you will find a simple table and some links to practice your
scraping skills.</p> <h2>Sample Table</h2> <table border="1"> <thead> <tr>
<th>Name</th> <th>Age</th> <th>City</th> </tr> </thead> <tbody> <tr> <td>Alice</td>
<td>30</td> <td>New York</td> </tr> <tr> <td>Bob</td> <td>25</td> <td>Los Angeles</td>
</tr> <tr> <td>Charlie</td> <td>35</td> <td>Chicago</td> </tr> </tbody> </table>
<h2>Useful Links</h2> <p>Check out the following resources for more information:</p> <ul>
<li><a href="https://www.python.org/">Python Official Website</a></li> <li><a
href="https://www.crummy.com/software/BeautifulSoup/">Beautiful Soup
Documentation</a></li> <li><a href="https://requests.readthedocs.io/">Requests: HTTP for
Humans</a></li> </ul></body></html>'

soup = BeautifulSoup(text, 'html.parser')

data = soup.find('p')

print(data.text)
```

```
Welcome to the web scraping demo page. Here, you will find a
simple table and some links to practice your scraping
skills.
```

Extract all paragraphs <p>

- find_all(“p”)

```
import requests
from bs4 import BeautifulSoup

text = '<!DOCTYPE html><html lang="en"><head> <meta charset="UTF-8"> <meta
name="viewport" content="width=device-width, initial-scale=1.0"> <title>Web Scraping
Demo</title></head><body> <h1>Web Scraping Demo Page</h1> <p>Welcome to the web
scraping demo page. Here, you will find a simple table and some links to practice your
scraping skills.</p> <h2>Sample Table</h2> <table border="1"> <thead> <tr>
<th>Name</th> <th>Age</th> <th>City</th> </tr> </thead> <tbody> <tr> <td>Alice</td>
<td>30</td> <td>New York</td> </tr> <tr> <td>Bob</td> <td>25</td> <td>Los Angeles</td>
</tr> <tr> <td>Charlie</td> <td>35</td> <td>Chicago</td> </tr> </tbody> </table>
<h2>Useful Links</h2> <p>Check out the following resources for more information:</p> <ul>
<li><a href="https://www.python.org/">Python Official Website</a></li> <li><a
href="https://www.crummy.com/software/BeautifulSoup/">Beautiful Soup
Documentation</a></li> <li><a href="https://requests.readthedocs.io/">Requests: HTTP for
Humans</a></li> </ul></body></html>'

soup = BeautifulSoup(text, 'html.parser')

data = soup.find_all('p')

for p in data:
    print(p.text)
```



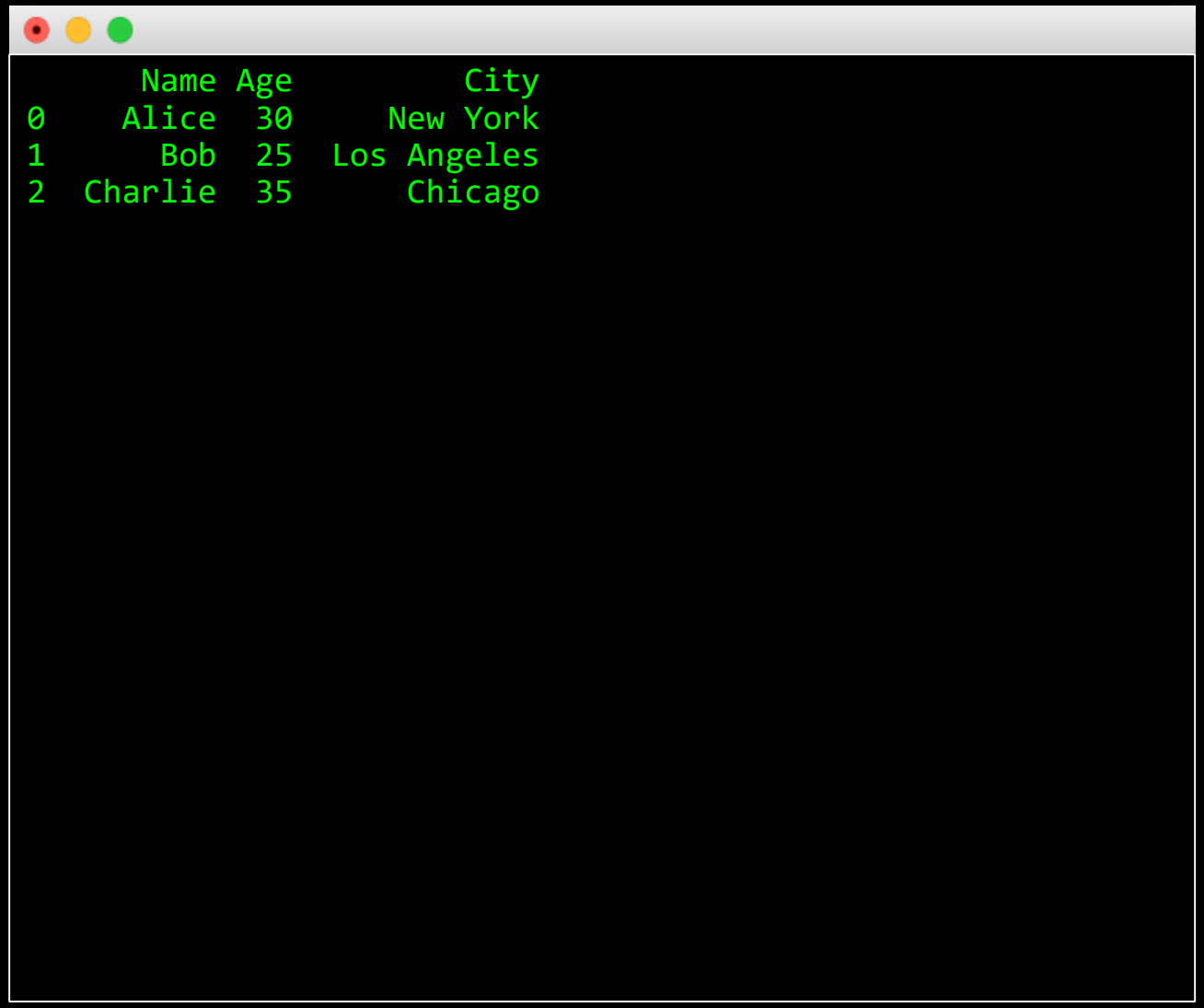
Extract all hyperlinks <a>

- find_all(“a”)

```
import requests
from bs4 import BeautifulSoup

text = '<!DOCTYPE html><html lang="en"><head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>Web Scraping Demo</title></head><body> <h1>Web Scraping Demo Page</h1> <p>Welcome to the web scraping demo page. Here, you will find a simple table and some links to practice your scraping skills.</p> <h2>Sample Table</h2> <table border="1"> <thead> <tr> <th>Name</th> <th>Age</th> <th>City</th> </tr> </thead> <tbody> <tr> <td>Alice</td> <td>30</td> <td>New York</td> </tr> <tr> <td>Bob</td> <td>25</td> <td>Los Angeles</td> </tr> <tr> <td>Charlie</td> <td>35</td> <td>Chicago</td> </tr> </tbody> </table> <h2>Useful Links</h2> <p>Check out the following resources for more information:</p> <ul> <li><a href="https://www.python.org/">Python Official Website</a></li> <li><a href="https://www.crummy.com/software/BeautifulSoup/">Beautiful Soup Documentation</a></li> <li><a href="https://requests.readthedocs.io/">Requests: HTTP for Humans</a></li> </ul></body></html>'

soup = BeautifulSoup(text, 'html.parser')

data = soup.find_all('a')

for a in data:
    print(a)
    print(a.text)
    print('\n')
```

```
<a href="https://www.python.org/">Python Official
Website</a>
Python Official Website


<a
href="https://www.crummy.com/software/BeautifulSoup/">Beauti
ful Soup Documentation</a>
Beautiful Soup Documentation


<a href="https://requests.readthedocs.io/">Requests: HTTP
for Humans</a>
Requests: HTTP for Humans
```

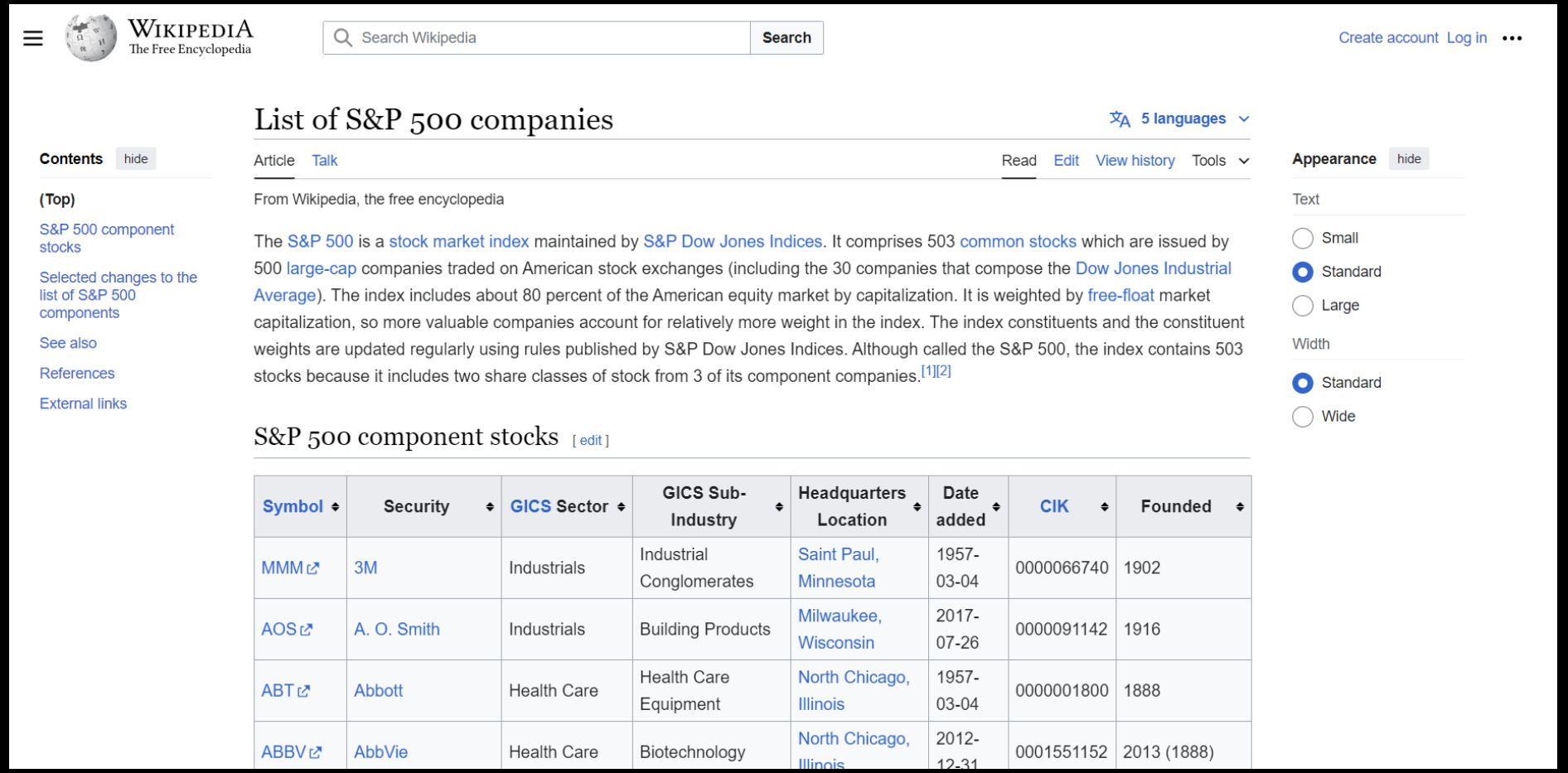
Extract all <table>



```
      Name Age         City
0    Alice  30     New York
1      Bob  25  Los Angeles
2  Charlie  35      Chicago
```

```
import requests
import pandas as pd
from bs4 import BeautifulSoup

text = '<!DOCTYPE html><html lang="en"><head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>Web Scraping Demo</title></head><body> <h1>Web Scraping Demo Page</h1> <p>Welcome to the web scraping demo page. Here, you will find a simple table and some links to practice your scraping skills.</p> <h2>Sample Table</h2> <table border="1"> <thead> <tr> <th>Name</th> <th>Age</th> <th>City</th> </tr> </thead> <tbody> <tr> <td>Alice</td> <td>30</td> <td>New York</td> </tr> <tr> <td>Bob</td> <td>25</td> <td>Los Angeles</td> </tr> <tr> <td>Charlie</td> <td>35</td> <td>Chicago</td> </tr> </tbody> </table> <h2>Useful Links</h2> <p>Check out the following resources for more information:</p> <ul> <li><a href="https://www.python.org/">Python Official Website</a></li> <li><a href="https://www.crummy.com/software/BeautifulSoup/">Beautiful Soup Documentation</a></li> <li><a href="https://requests.readthedocs.io/">Requests: HTTP for Humans</a></li> </ul></body></html>'

soup = BeautifulSoup(text, 'html.parser')

# Find the table
table = soup.find('table')

# Extract the headers
headers = []
for th in table.find_all('th'):
    headers.append(th.text)

# Extract the rows
rows = []
for tr in table.find_all('tr')[1:]: # Skip the header row
    cells = tr.find_all('td')
    row = [cell.text for cell in cells]
    rows.append(row)

# Create a DataFrame
df = pd.DataFrame(rows, columns=headers)
print(df)
```

Scrape S&P 500 stock composites



WIKIPEDIA
The Free Encyclopedia

Search Wikipedia | Search

Create account | Log in

List of S&P 500 companies

5 languages

Article | Talk | Read | Edit | View history | Tools

From Wikipedia, the free encyclopedia

Contents hide

(Top)
S&P 500 component stocks
Selected changes to the list of S&P 500 components
See also
References
External links

Appearance hide

Text
Small
Standard
Large

Width
Standard
Wide

The S&P 500 is a stock market index maintained by S&P Dow Jones Indices. It comprises 503 common stocks which are issued by 500 large-cap companies traded on American stock exchanges (including the 30 companies that compose the Dow Jones Industrial Average). The index includes about 80 percent of the American equity market by capitalization. It is weighted by free-float market capitalization, so more valuable companies account for relatively more weight in the index. The index constituents and the constituent weights are updated regularly using rules published by S&P Dow Jones Indices. Although called the S&P 500, the index contains 503 stocks because it includes two share classes of stock from 3 of its component companies.[1][2]

S&P 500 component stocks [edit]

Symbol	Security	GICS Sector	GICS Sub-Industry	Headquarters Location	Date added	CIK	Founded
MMM	3M	Industrials	Industrial Conglomerates	Saint Paul, Minnesota	1957-03-04	0000066740	1902
AOS	A. O. Smith	Industrials	Building Products	Milwaukee, Wisconsin	2017-07-26	0000091142	1916
ABT	Abbott	Health Care	Health Care Equipment	North Chicago, Illinois	1957-03-04	0000001800	1888
ABBV	AbbVie	Health Care	Biotechnology	North Chicago, Illinois	2012-12-31	0001551152	2013 (1888)

https://en.wikipedia.org/wiki/List_of_S%26P_500_companies

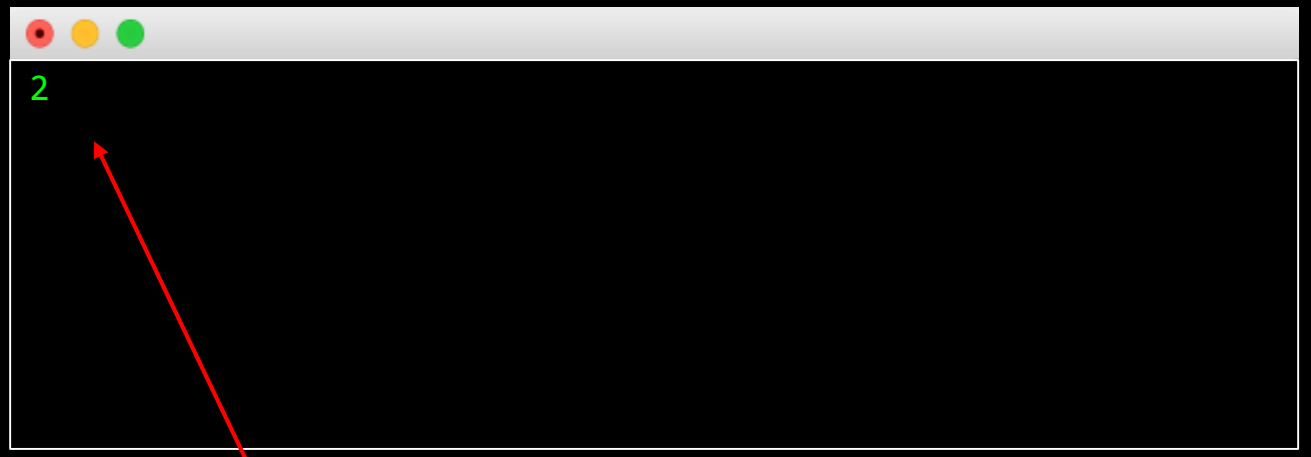
Get HTML content of the website

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

url = 'https://en.wikipedia.org/wiki/List_of_S%26P_500_companies'
text = requests.get(url).text

print(text)
```

```
Please set a user-agent and respect our robot policy
https://w.wiki/4wJS. See also
https://phabricator.wikimedia.org/T400119.
```

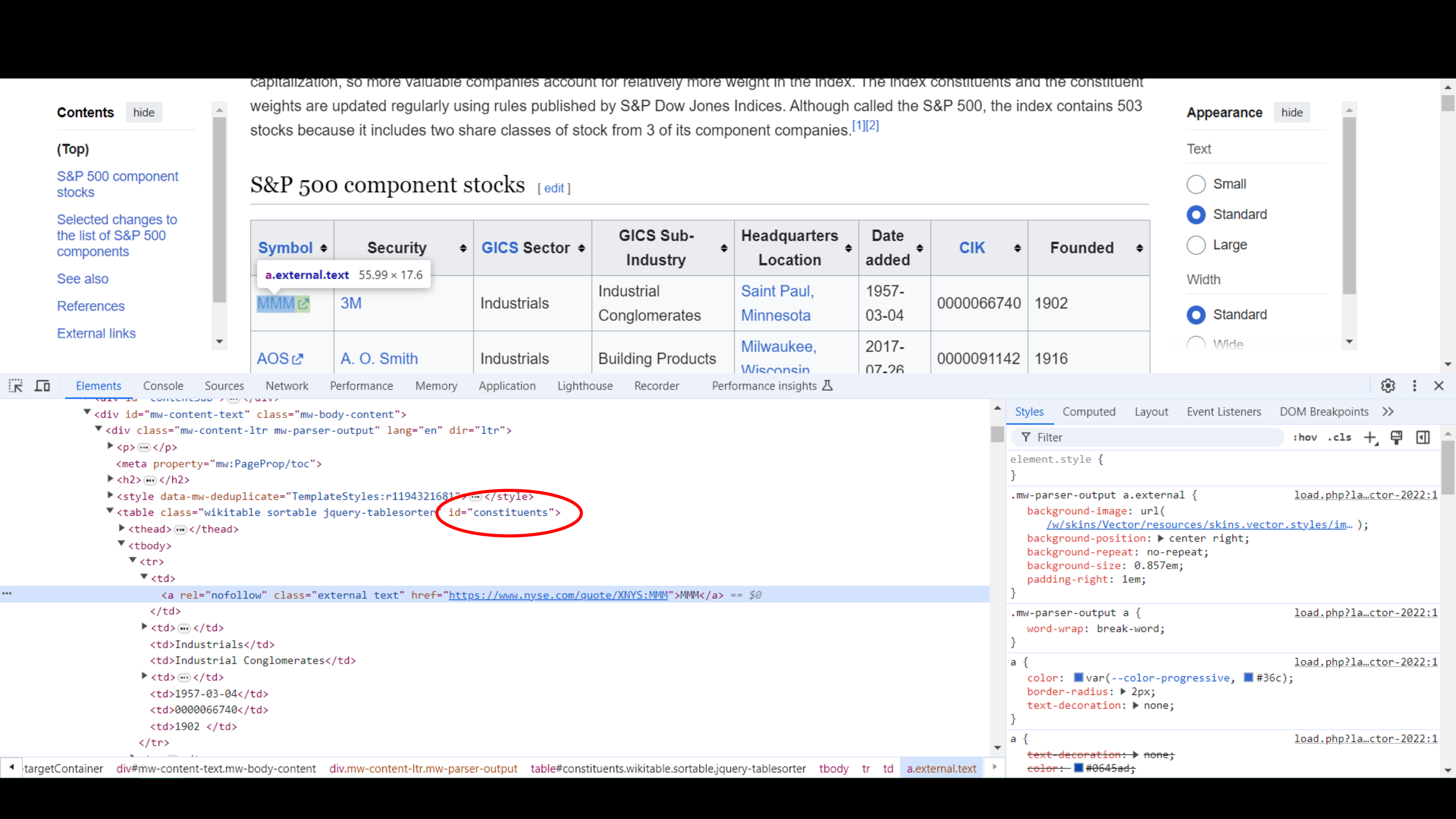
Why we can’t get the result?

Extract all <table>



There are 2 tables in the wiki page.
How to locate the correct table?

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

headers = {
   "user-agent": "Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/143.0.0.0 Safari/537.36"
}

url =
'https://en.wikipedia.org/wiki/List_of_S%26P_500_com
panies'
text = requests.get(url, headers=headers).text

soup = BeautifulSoup(text, 'html.parser')

data = soup.find_all('table')
print(len(data))
```


Contents hide
(Top)
S&P 500 component stocks
Selected changes to the list of S&P 500 components
See also
References
External links
capitalization, so more valuable companies account for relatively more weight in the index. The index constituents and the constituent weights are updated regularly using rules published by S&P Dow Jones Indices. Although called the S&P 500, the index contains 503 stocks because it includes two share classes of stock from 3 of its component companies.[1][2]
S&P 500 component stocks [edit]
Symbol
Security
GICS Sector
GICS Sub-Industry
Headquarters Location
Date added
CIK
Founded
a.external.text 55.99 × 17.6
MMM
3M
Industrials
Industrial Conglomerates
Saint Paul, Minnesota
1957-03-04
0000066740
1902
AOS
A. O. Smith
Industrials
Building Products
Milwaukee, Wisconsin
2017-07-26
0000091142
1916
Appearance hide
Text
Small
Standard
Large
Width
Standard
Wide
Elements Console Sources Network Performance Memory Application Lighthouse Recorder Performance insights

…
<meta property="mw:PageProp/toc">
<h2>…</h2>
<style data-mw-deduplicate="TemplateStyles:r1194321681">…</style>
<table class="wikitable sortable jquery-tablesorter" id="constituents">
<thead>…</thead>
<tbody>
<tr>
<td>
MMM == $0
</td>
<td>…</td>
<td>Industrials</td>
<td>Industrial Conglomerates</td>
<td>…</td>
<td>1957-03-04</td>
<td>0000066740</td>
<td>1902 </td>
</tr>
Styles Computed Layout Event Listeners DOM Breakpoints
Filter :hov .cls
element.style {
}
.mw-parser-output a.external { load.php?la…ctor-2022:1
background-image: url(/w/skins/Vector/resources/skins.vector.styles/im…);
background-position: center right;
background-repeat: no-repeat;
background-size: 0.857em;
padding-right: 1em;
}
.mw-parser-output a { load.php?la…ctor-2022:1
word-wrap: break-word;
}
a { load.php?la…ctor-2022:1
color: var(--color-progressive, #36c);
border-radius: 2px;
text-decoration: none;
}
a { load.php?la…ctor-2022:1
text-decoration: none;
color: #0645ad;
targetContainer div#mw-content-text.mw-body-content div.mw-content-ltr.mw-parser-output table#constituents.wikitable.sortable.jquery-tablesorter tbody tr td a.external.text

Extract all <table>

	Symbol	Security	GICS Sector	...	Date added	CIK	Founded
0	MMM	3M	Industrials	...	1957-03-04	0000066740	1902
1	AOS	A. O. Smith	Industrials	...	2017-07-26	0000091142	1916
2	ABT	Abbott	Health Care	...	1957-03-04	0000001800	1888
3	ABBV	AbbVie	Health Care	...	2012-12-31	0001551152	2013 (1888)
4	ACN	Accenture	Information Technology	...	2011-07-06	0001467373	1989
..	...	...	...	...	...	...	...
498	XYL	Xylem Inc.	Industrials	...	2011-11-01	0001524472	2011
499	YUM	Yum! Brands	Consumer Discretionary	...	1997-10-06	0001041061	1997
500	ZBRA	Zebra Technologies	Information Technology	...	2019-12-23	0000877212	1969
501	ZBH	Zimmer Biomet	Health Care	...	2001-08-07	0001136869	1927
502	ZTS	Zoetis	Health Care	...	2013-06-21	0001555280	1952

```python
import requests
from bs4 import BeautifulSoup
import pandas as pd

headers = {
  "user-agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0
Safari/537.36"
}

url = 'https://en.wikipedia.org/wiki/List_of_S%26P_500_companies'
text = requests.get(url, headers=headers).text


soup = BeautifulSoup(text, 'html.parser')

# Find the table
table = soup.find('table', id="constituents")

# Extract the headers
headers = []
for th in table.find_all('th'):
    headers.append(th.text.strip() )

# Extract the rows
rows = []
for tr in table.find_all('tr')[1:]: # Skip the header row
    cells = tr.find_all('td')
    row = [cell.text.strip() for cell in cells]
    rows.append(row)

# Create a DataFrame
df = pd.DataFrame(rows, columns=headers)
print(df)
```

Other search methods

- Search by CSS class

```
table = soup.find_all('table', class_='wikitable sortable')
```

- Search by tag attributes

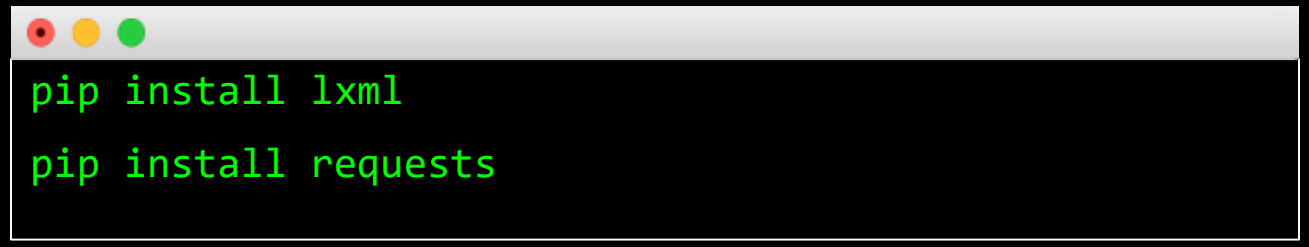
```
table = soup.find_all("table", attrs={"class": "wikitable sortable"})
```

- CSS Selector

```
table = soup.select(‘table')
table = soup.select('table[class="wikitable sortable"]')
table = soup.select('.wikitable')
table = soup.select('.wikitable.sortable')
```

LXML

- Popular python library for parsing XPath and XML schema
- Official Doc: https://lxml.de/



What is XPath?

- XPath is a query language to navigate the elements and attributes in an XML document
- Key features:
  - **Node Selection**: XPath allows you to select nodes or elements in an XML document. For example, selecting elements by their tag name, attribute, or position.
  - **Path Expressions**: Provides a way to navigate through elements and attributes in an XML document using path expressions.
  - **Syntax**: Uses a compact, non-XML syntax to make it easier to write and read.

Common XPath Syntax

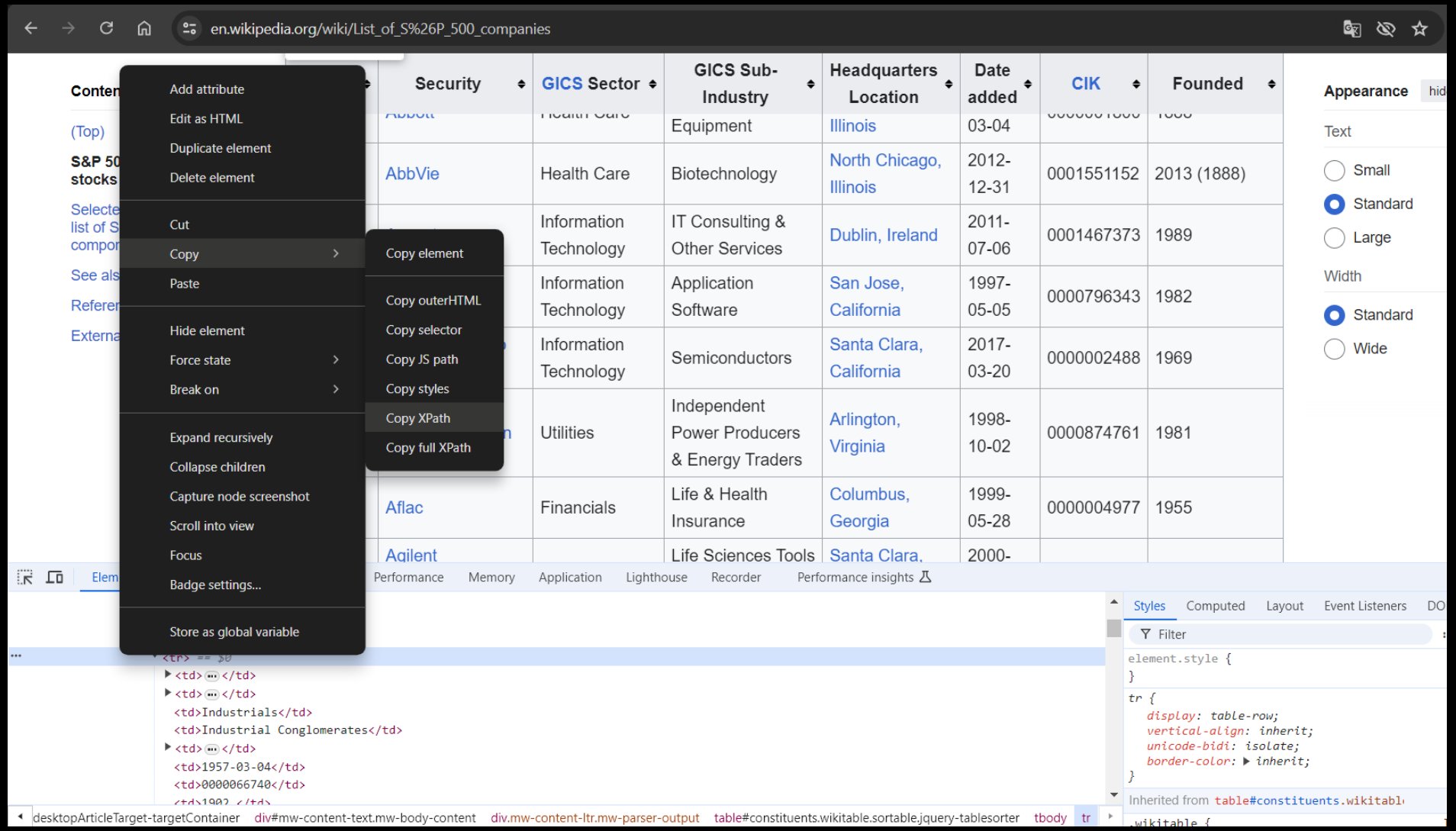
- **Basic Path**: /root/child
  - Selects the child element of the root element.
- **Relative Path**: //child
  - Selects all child elements in the document.
- **Attributes**: //element[@attribute='value']
  - Selects elements with a specific attribute value.
- **Functions**: //element[position()=1]
  - Selects the first element.

XPath Example

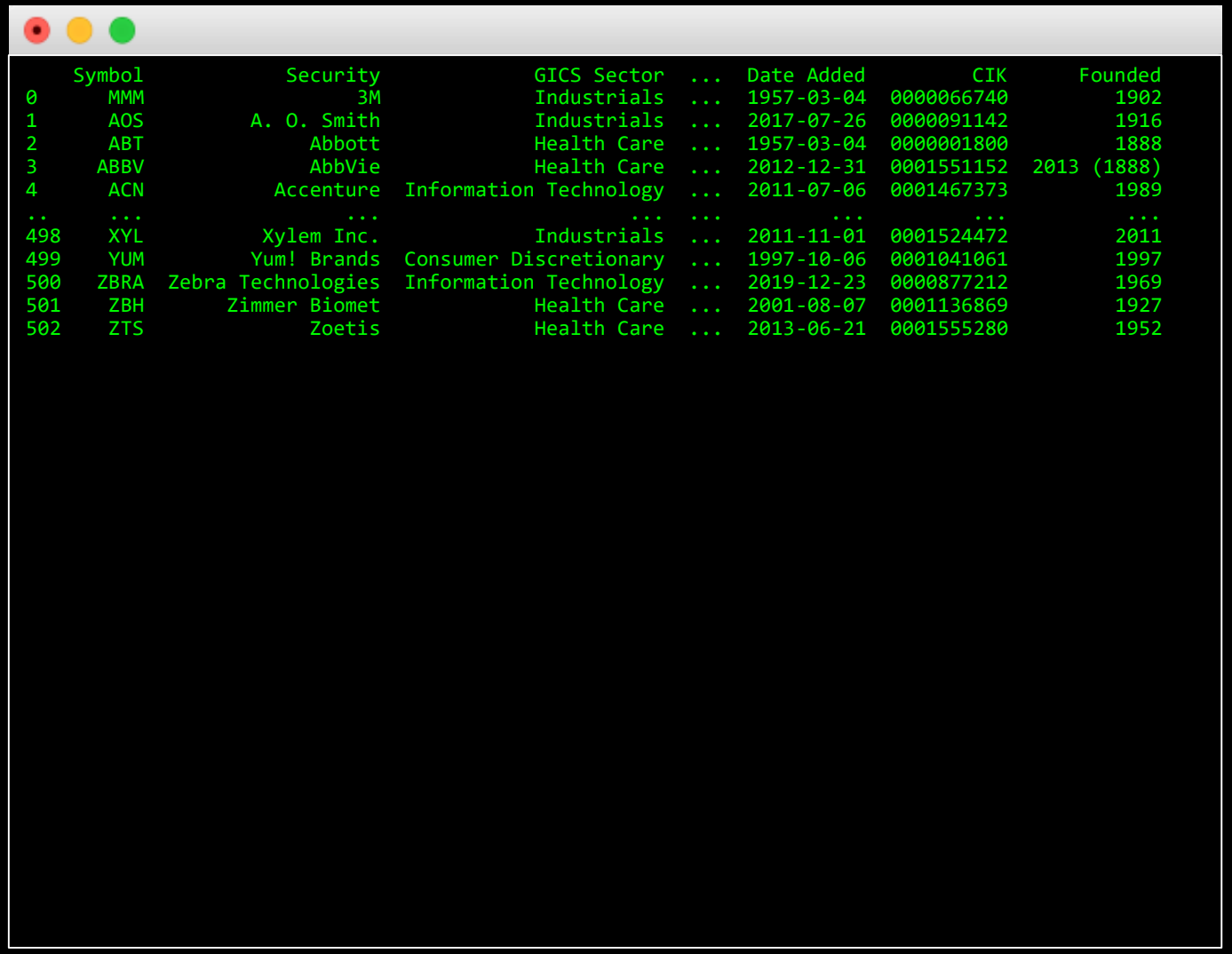
- **Select all <li> elements**: //li
- **Select <li> elements with class "item"**: //li[@class='item']
- **Select the second <li> element**: //li[2]

```
<ul>
    <li class="item">Item 1</li>
    <li class="item">Item 2</li>
    <li class="item">Item 3</li>
</ul>
```

//*[@id="constituents"]/tbody/tr[1]



Extract stock list



	Symbol	Security	GICS Sector	...	Date Added	CIK	Founded
0	MMM	3M	Industrials	...	1957-03-04	0000066740	1902
1	AOS	A. O. Smith	Industrials	...	2017-07-26	0000091142	1916
2	ABT	Abbott	Health Care	...	1957-03-04	0000001800	1888
3	ABBV	AbbVie	Health Care	...	2012-12-31	0001551152	2013 (1888)
4	ACN	Accenture	Information Technology	...	2011-07-06	0001467373	1989
..	...	...	...	...	...	...	...
498	XYL	Xylem Inc.	Industrials	...	2011-11-01	0001524472	2011
499	YUM	Yum! Brands	Consumer Discretionary	...	1997-10-06	0001041061	1997
500	ZBRA	Zebra Technologies	Information Technology	...	2019-12-23	0000877212	1969
501	ZBH	Zimmer Biomet	Health Care	...	2001-08-07	0001136869	1927
502	ZTS	Zoetis	Health Care	...	2013-06-21	0001555280	1952

```python
from lxml import html
import requests
import pandas as pd

headers = {
  "user-agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36"
}

# download web page
url = 'https://en.wikipedia.org/wiki/List_of_S%26P_500_companies'
response = requests.get(url, headers=headers )

# Check that the request was successful
text = response.content

# load into html tree
tree = html.fromstring(text)

# extract table rows from XPath
table_rows = tree.xpath('//*[@id="constituents"]/tbody/tr')

# parse the table rows
data = []
for row in table_rows[1:]: # Skip header row
    cells = row.xpath('td')
    if len(cells) > 0:
        try:
            symbol = cells[0].text_content().strip()
            security = cells[1].text_content().strip()
            gics_sector = cells[2].text_content().strip()
            gics_sub_industry = cells[3].text_content().strip()
            headquarters = cells[4].text_content().strip()
            date_added = cells[5].text_content().strip()
            cik = cells[6].text_content().strip()
            founded = cells[7].text_content().strip()
            data.append([symbol, security, gics_sector, gics_sub_industry, headquarters, date_added, cik, founded])
        except Exception as e:
            pass

# create a DataFrame
columns = ['Symbol', 'Security', 'GICS Sector', 'GICS Sub-Industry', 'Headquarters Location', 'Date Added', 'CIK', 'Founded']
df = pd.DataFrame(data, columns=columns)

# print the DataFrame
print(df)
```

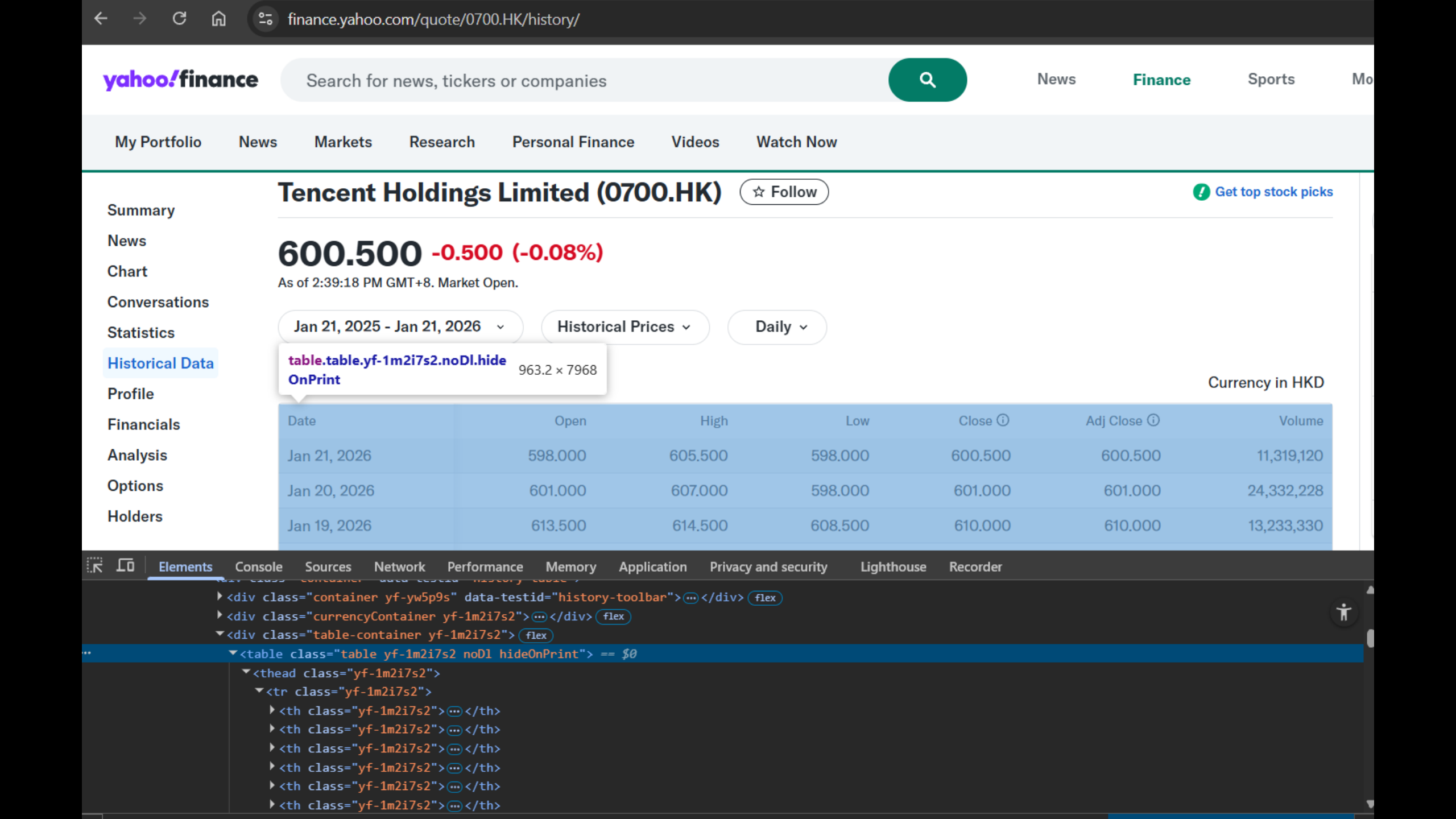
Yahoo Finance

Yahoo Finance (https://finance.yahoo.com/)

Available datasets

- Market data
  - Stock, ETF, Index
  - Historical data (End-of-day, weekly, monthly)
  - Real-time data (up to 20 min delay)
- Corporate actions (eg. dividends, splits)
- Company Profile (eg. Industry, sector, key executives, etc)
- Financial (eg. Income statement, Balance sheet, Cash flow)
- Analyst Estimates (earnings, revenue, growth)

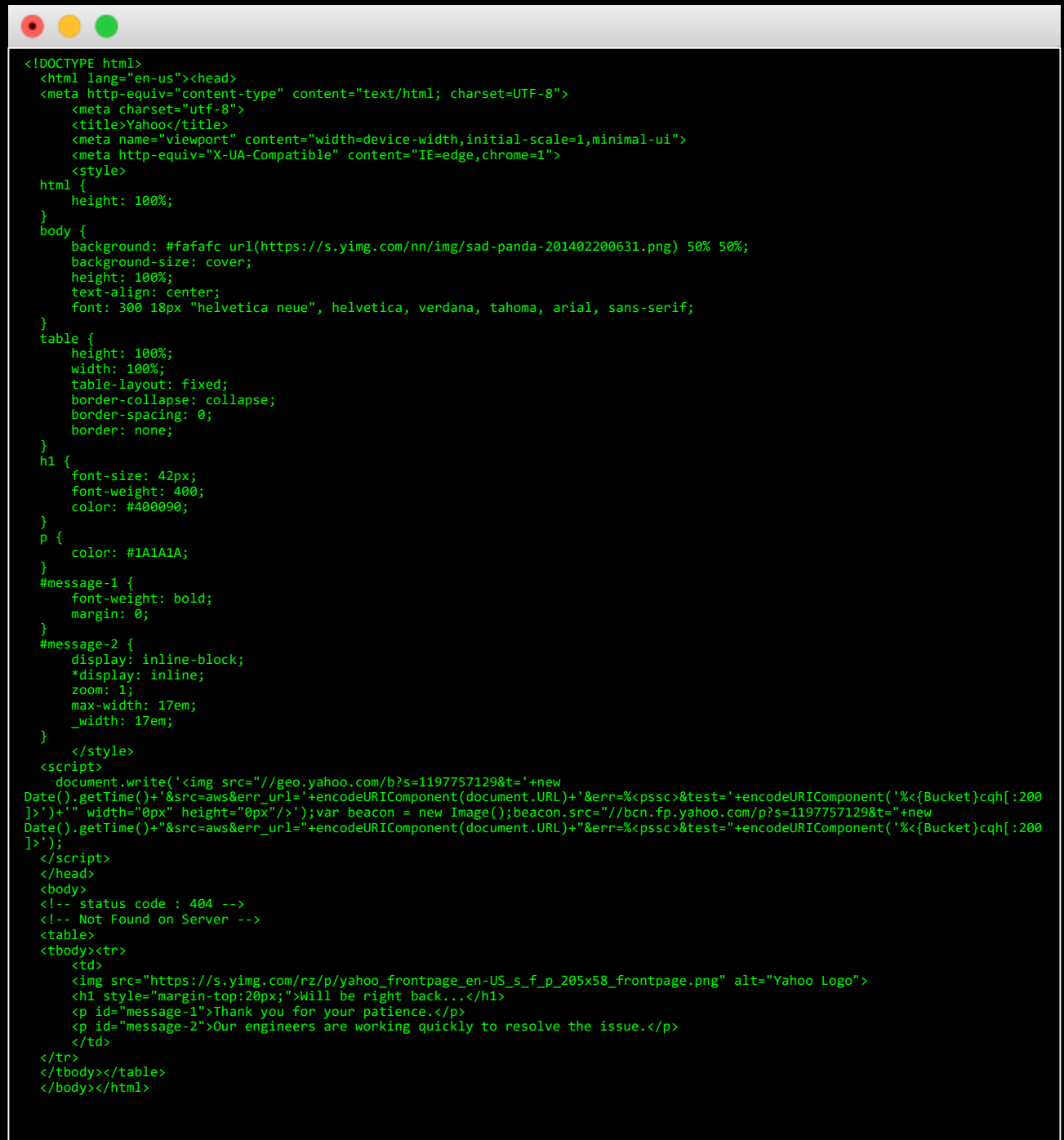
It's FREE ☺


finance.yahoo.com/quote/0700.HK/history/
yahoo!finance
Search for news, tickers or companies
News
Finance
Sports
My Portfolio
News
Markets
Research
Personal Finance
Videos
Watch Now
Summary
News
Chart
Conversations
Statistics
Historical Data
Profile
Financials
Analysis
Options
Holders
Tencent Holdings Limited (0700.HK)
Follow
Get top stock picks
600.500 -0.500 (-0.08%)
As of 2:39:18 PM GMT+8. Market Open.
Jan 21, 2025 - Jan 21, 2026
Historical Prices
Daily
table.table.yf-1m2i7s2.noDl.hideOnPrint
963.2 × 7968
Currency in HKD
Date
Open
High
Low
Close
Adj Close
Volume
Jan 21, 2026
598.000
605.500
598.000
600.500
600.500
11,319,120
Jan 20, 2026
601.000
607.000
598.000
601.000
601.000
24,332,228
Jan 19, 2026
613.500
614.500
608.500
610.000
610.000
13,233,330
Elements
Console
Sources
Network
Performance
Memory
Application
Privacy and security
Lighthouse
Recorder

<table class="table yf-1m2i7s2 noDl hideOnPrint"> == $0
<thead class="yf-1m2i7s2">
<tr class="yf-1m2i7s2">
<th class="yf-1m2i7s2">
<th class="yf-1m2i7s2">
<th class="yf-1m2i7s2">
<th class="yf-1m2i7s2">
<th class="yf-1m2i7s2">
<th class="yf-1m2i7s2">
<th class="yf-1m2i7s2">
flex
flex
flex

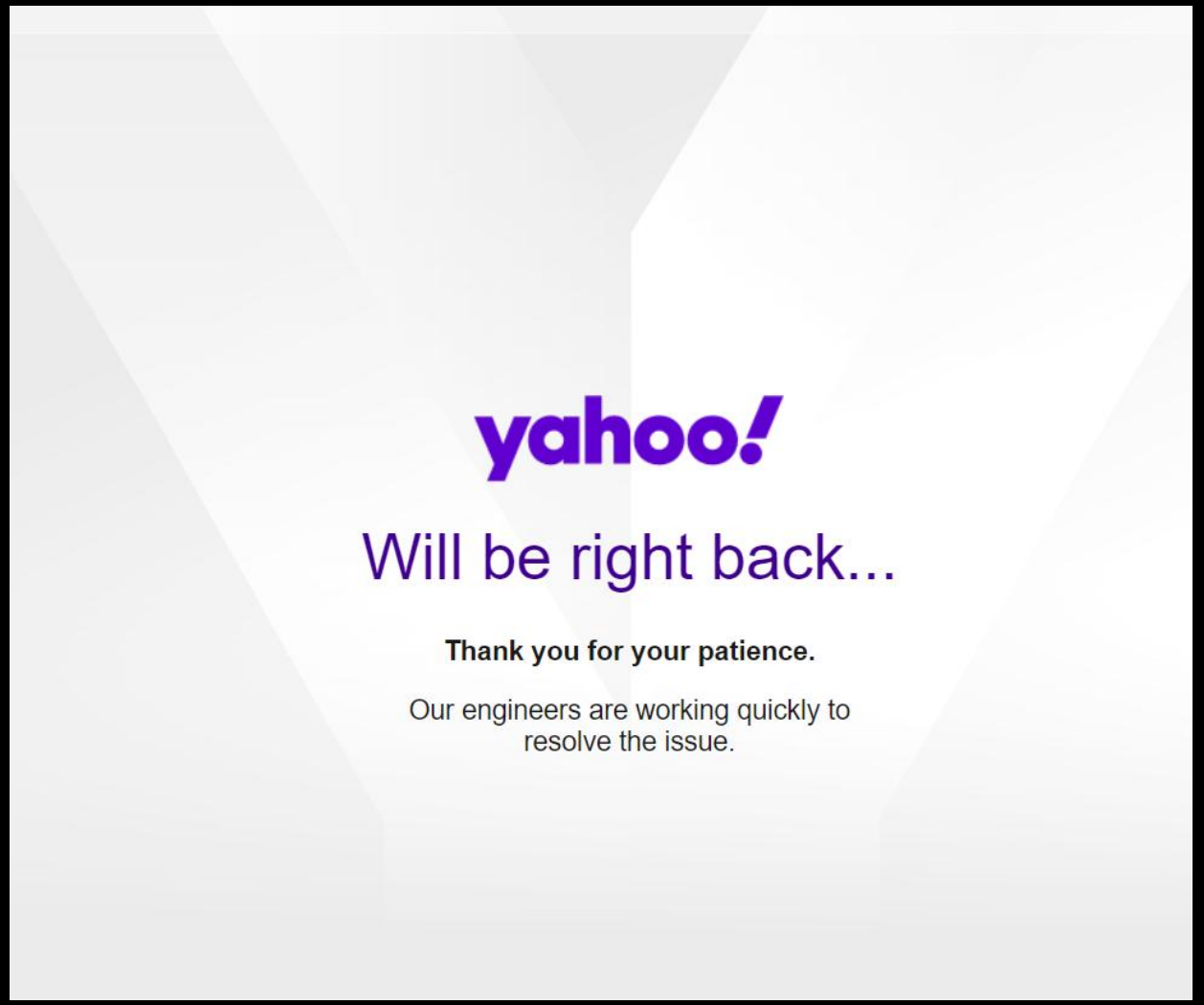
Extract stock price

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

url = 'https://finance.yahoo.com/quote/0700.HK/history/'
text = requests.get(url).text

print(text)

soup = BeautifulSoup(text, 'html.parser')

# Find the table
table - soup.find('table', class_='table yf-1m2i7s2')

# Extract the headers
headers = []
for th in table.find_all('th'):
    headers.append(th.text.strip())

# Extract the rows
rows = []
for tr in table.find_all('tr')[1:]: # Skip the header row
    cells = tr.find_all('td')
    row = [cell.text.strip() for cell in cells]
    rows.append(row)

# Create a DataFrame
df = pd.DataFrame(rows, columns=headers)
print(df)
```



```
<!DOCTYPE html>
  <html lang="en-us"><head>
  <meta http-equiv="content-type" content="text/html; charset=UTF-8">
      <meta charset="utf-8">
      <title>Yahoo</title>
      <meta name="viewport" content="width=device-width,initial-scale=1,minimal-ui">
      <meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1">
      <style>
  html {
      height: 100%;
  }
  body {
      background: #fafafc url(https://s.yimg.com/nn/img/sad-panda-201402200631.png) 50% 50%;
      background-size: cover;
      height: 100%;
      text-align: center;
      font: 300 18px "helvetica neue", helvetica, verdana, tahoma, arial, sans-serif;
  }
  table {
      height: 100%;
      width: 100%;
      table-layout: fixed;
      border-collapse: collapse;
      border-spacing: 0;
      border: none;
  }
  h1 {
      font-size: 42px;
      font-weight: 400;
      color: #400090;
  }
  p {
      color: #1A1A1A;
  }
  #message-1 {
      font-weight: bold;
      margin: 0;
  }
  #message-2 {
      display: inline-block;
      *display: inline;
      zoom: 1;
      max-width: 17em;
      _width: 17em;
  }
      </style>
  <script>
    document.write('<img src="//geo.yahoo.com/b?s=1197757129&t='+new
Date().getTime()+'&src=aws&err_url='+encodeURIComponent(document.URL)+'&err=%<pssc>&test='+encodeURIComponent('%<{Bucket}cqh[:200
]>')+'" width="0px" height="0px"/>');var beacon = new Image();beacon.src="//bcn.fp.yahoo.com/p?s=1197757129&t="+new
Date().getTime()+"&src=aws&err_url="+encodeURIComponent(document.URL)+"&err=%<pssc>&test="+encodeURIComponent('%<{Bucket}cqh[:200
]>');
  </script>
  </head>
  <body>
  <!-- status code : 404 -->
  <!-- Not Found on Server -->
  <table>
  <tbody><tr>
      <td>
      <img src="https://s.yimg.com/rz/p/yahoo_frontpage_en-US_s_f_p_205x58_frontpage.png" alt="Yahoo Logo">
      <h1 style="margin-top:20px;">Will be right back...</h1>
      <p id="message-1">Thank you for your patience.</p>
      <p id="message-2">Our engineers are working quickly to resolve the issue.</p>
      </td>
  </tr>
  </tbody></table>
  </body></html>
```

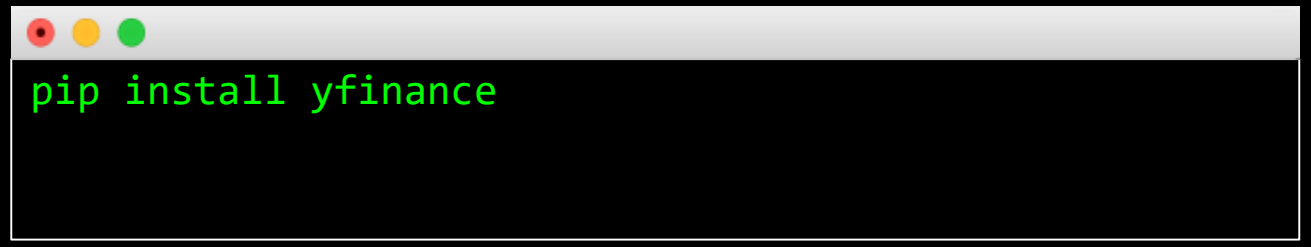
Why it doesn’t work?



???

yfinance

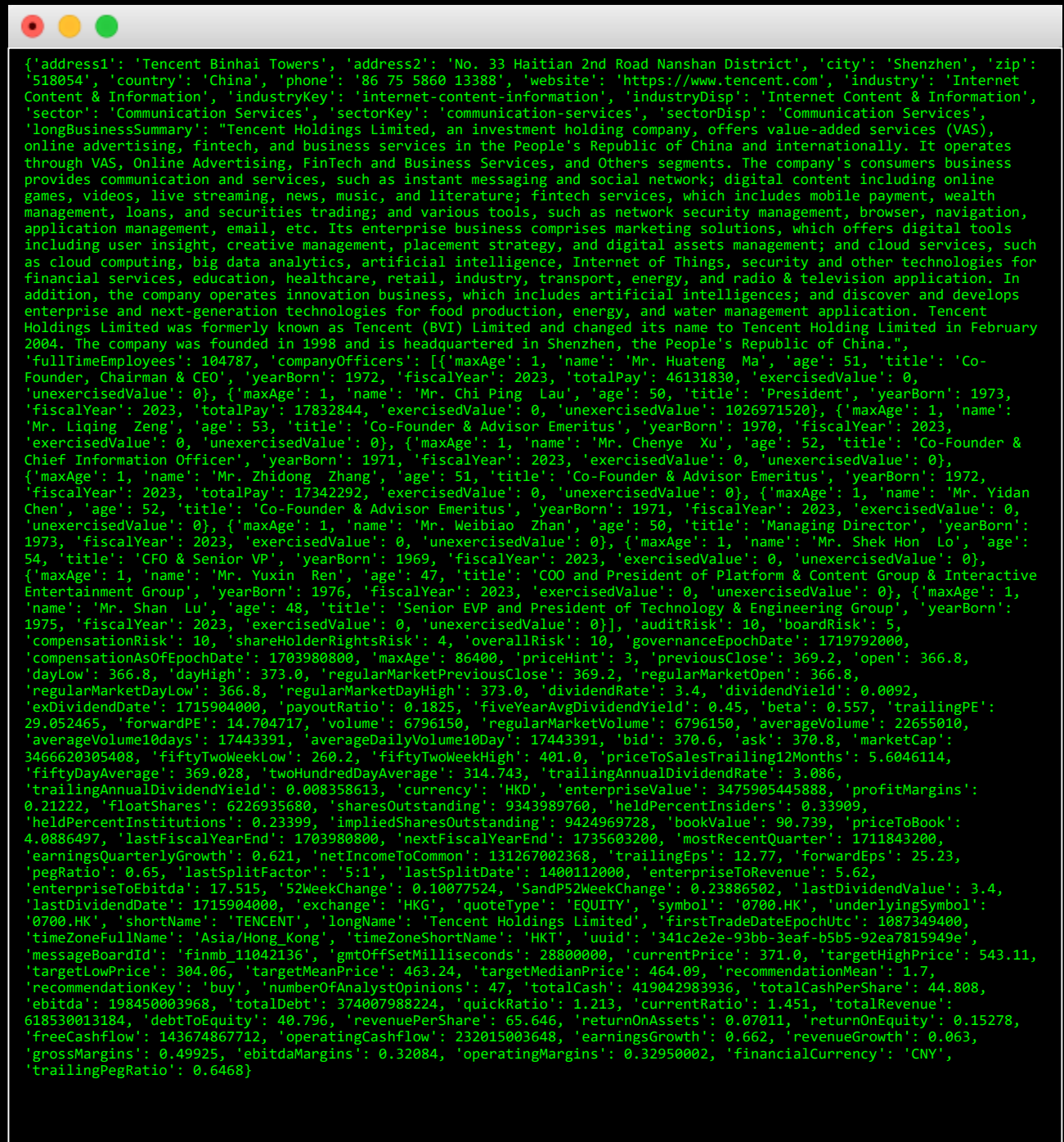
- It is an unofficial library for Yahoo Finance
- Project website: https://pypi.org/project/yfinance/



Extract stock info

- .info()

```
import yfinance as yf

ticker = yf.Ticker("0700.HK")

# get ticker basic info
print(ticker.info)
```



{'address1': 'Tencent Binhai Towers', 'address2': 'No. 33 Haitian 2nd Road Nanshan District', 'city': 'Shenzhen', 'zip': '518054', 'country': 'China', 'phone': '86 75 5860 13388', 'website': 'https://www.tencent.com', 'industry': 'Internet Content & Information', 'industryKey': 'internet-content-information', 'industryDisp': 'Internet Content & Information', 'sector': 'Communication Services', 'sectorKey': 'communication-services', 'sectorDisp': 'Communication Services', 'longBusinessSummary': "Tencent Holdings Limited, an investment holding company, offers value-added services (VAS), online advertising, fintech, and business services in the People's Republic of China and internationally. It operates through VAS, Online Advertising, FinTech and Business Services, and Others segments. The company's consumers business provides communication and services, such as instant messaging and social network; digital content including online games, videos, live streaming, news, music, and literature; fintech services, which includes mobile payment, wealth management, loans, and securities trading; and various tools, such as network security management, browser, navigation, application management, email, etc. Its enterprise business comprises marketing solutions, which offers digital tools including user insight, creative management, placement strategy, and digital assets management; and cloud services, such as cloud computing, big data analytics, artificial intelligence, Internet of Things, security and other technologies for financial services, education, healthcare, retail, industry, transport, energy, and radio & television application. In addition, the company operates innovation business, which includes artificial intelligences; and discover and develops enterprise and next-generation technologies for food production, energy, and water management application. Tencent Holdings Limited was formerly known as Tencent (BVI) Limited and changed its name to Tencent Holding Limited in February 2004. The company was founded in 1998 and is headquartered in Shenzhen, the People's Republic of China.", 'fullTimeEmployees': 104787, 'companyOfficers': [{'maxAge': 1, 'name': 'Mr. Huateng Ma', 'age': 51, 'title': 'Co-Founder, Chairman & CEO', 'yearBorn': 1972, 'fiscalYear': 2023, 'totalPay': 46131830, 'exercisedValue': 0, 'unexercisedValue': 0}, {'maxAge': 1, 'name': 'Mr. Chi Ping Lau', 'age': 50, 'title': 'President', 'yearBorn': 1973, 'fiscalYear': 2023, 'totalPay': 17832844, 'exercisedValue': 0, 'unexercisedValue': 1026971520}, {'maxAge': 1, 'name': 'Mr. Liqing Zeng', 'age': 53, 'title': 'Co-Founder & Advisor Emeritus', 'yearBorn': 1970, 'fiscalYear': 2023, 'exercisedValue': 0, 'unexercisedValue': 0}, {'maxAge': 1, 'name': 'Mr. Chenye Xu', 'age': 52, 'title': 'Co-Founder & Chief Information Officer', 'yearBorn': 1971, 'fiscalYear': 2023, 'exercisedValue': 0, 'unexercisedValue': 0}, {'maxAge': 1, 'name': 'Mr. Zhidong Zhang', 'age': 51, 'title': 'Co-Founder & Advisor Emeritus', 'yearBorn': 1972, 'fiscalYear': 2023, 'totalPay': 17342292, 'exercisedValue': 0, 'unexercisedValue': 0}, {'maxAge': 1, 'name': 'Mr. Yidan Chen', 'age': 52, 'title': 'Co-Founder & Advisor Emeritus', 'yearBorn': 1971, 'fiscalYear': 2023, 'exercisedValue': 0, 'unexercisedValue': 0}, {'maxAge': 1, 'name': 'Mr. Weibiao Zhan', 'age': 50, 'title': 'Managing Director', 'yearBorn': 1973, 'fiscalYear': 2023, 'exercisedValue': 0, 'unexercisedValue': 0}, {'maxAge': 1, 'name': 'Mr. Shek Hon Lo', 'age': 54, 'title': 'CFO & Senior VP', 'yearBorn': 1969, 'fiscalYear': 2023, 'exercisedValue': 0, 'unexercisedValue': 0}, {'maxAge': 1, 'name': 'Mr. Yuxin Ren', 'age': 47, 'title': 'COO and President of Platform & Content Group & Interactive Entertainment Group', 'yearBorn': 1976, 'fiscalYear': 2023, 'exercisedValue': 0, 'unexercisedValue': 0}, {'maxAge': 1, 'name': 'Mr. Shan Lu', 'age': 48, 'title': 'Senior EVP and President of Technology & Engineering Group', 'yearBorn': 1975, 'fiscalYear': 2023, 'exercisedValue': 0, 'unexercisedValue': 0}], 'auditRisk': 10, 'boardRisk': 5, 'compensationRisk': 10, 'shareHolderRightsRisk': 4, 'overallRisk': 10, 'governanceEpochDate': 1719792000, 'compensationAsOfEpochDate': 1703980800, 'maxAge': 86400, 'priceHint': 3, 'previousClose': 369.2, 'open': 366.8, 'dayLow': 366.8, 'dayHigh': 373.0, 'regularMarketPreviousClose': 369.2, 'regularMarketOpen': 366.8, 'regularMarketDayLow': 366.8, 'regularMarketDayHigh': 373.0, 'dividendRate': 3.4, 'dividendYield': 0.0092, 'exDividendDate': 1715904000, 'payoutRatio': 0.1825, 'fiveYearAvgDividendYield': 0.45, 'beta': 0.557, 'trailingPE': 29.052465, 'forwardPE': 14.704717, 'volume': 6796150, 'regularMarketVolume': 6796150, 'averageVolume': 22655010, 'averageVolume10days': 17443391, 'averageDailyVolume10Day': 17443391, 'bid': 370.6, 'ask': 370.8, 'marketCap': 3466620305408, 'fiftyTwoWeekLow': 260.2, 'fiftyTwoWeekHigh': 401.0, 'priceToSalesTrailing12Months': 5.6046114, 'fiftyDayAverage': 369.028, 'twoHundredDayAverage': 314.743, 'trailingAnnualDividendRate': 3.086, 'trailingAnnualDividendYield': 0.008358613, 'currency': 'HKD', 'enterpriseValue': 3475905445888, 'profitMargins': 0.21222, 'floatShares': 6226935680, 'sharesOutstanding': 9343989760, 'heldPercentInsiders': 0.33909, 'heldPercentInstitutions': 0.23399, 'impliedSharesOutstanding': 9424969728, 'bookValue': 90.739, 'priceToBook': 4.0886497, 'lastFiscalYearEnd': 1703980800, 'nextFiscalYearEnd': 1735603200, 'mostRecentQuarter': 1711843200, 'earningsQuarterlyGrowth': 0.621, 'netIncomeToCommon': 131267002368, 'trailingEps': 12.77, 'forwardEps': 25.23, 'pegRatio': 0.65, 'lastSplitFactor': '5:1', 'lastSplitDate': 1400112000, 'enterpriseToRevenue': 5.62, 'enterpriseToEbitda': 17.515, '52WeekChange': 0.10077524, 'SandP52WeekChange': 0.23886502, 'lastDividendValue': 3.4, 'lastDividendDate': 1715904000, 'exchange': 'HKG', 'quoteType': 'EQUITY', 'symbol': '0700.HK', 'underlyingSymbol': '0700.HK', 'shortName': 'TENCENT', 'longName': 'Tencent Holdings Limited', 'firstTradeDateEpochUtc': 1087349400, 'timeZoneFullName': 'Asia/Hong_Kong', 'timeZoneShortName': 'HKT', 'uuid': '341c2e2e-93bb-3eaf-b5b5-92ea7815949e', 'messageBoardId': 'finmb_11042136', 'gmtOffSetMilliseconds': 28800000, 'currentPrice': 371.0, 'targetHighPrice': 543.11, 'targetLowPrice': 304.06, 'targetMeanPrice': 463.24, 'targetMedianPrice': 464.09, 'recommendationMean': 1.7, 'recommendationKey': 'buy', 'numberOfAnalystOpinions': 47, 'totalCash': 419042983936, 'totalCashPerShare': 44.808, 'ebitda': 198450003968, 'totalDebt': 374007988224, 'quickRatio': 1.213, 'currentRatio': 1.451, 'totalRevenue': 618530013184, 'debtToEquity': 40.796, 'revenuePerShare': 65.646, 'returnOnAssets': 0.07011, 'returnOnEquity': 0.15278, 'freeCashflow': 143674867712, 'operatingCashflow': 232015003648, 'earningsGrowth': 0.662, 'revenueGrowth': 0.063, 'grossMargins': 0.49925, 'ebitdaMargins': 0.32084, 'operatingMargins': 0.32950002, 'financialCurrency': 'CNY', 'trailingPegRatio': 0.6468}

Extract stock history

- .history(period)
- available period ['1d', '5d', '1mo', '3mo', '6mo', '1y', '2y', '5y', '10y', 'ytd', 'max']

```python
import yfinance as yf

ticker = yf.Ticker("0700.HK")

# download historical data
data = ticker.history(period='1mo')

print(data)
```

Date	Open	High	Low	Close	Volume	Dividends	Stock Splits
2025-12-22 00:00:00+08:00	620.0	621.5	610.0	614.5	13868060	0.0	0.0
2025-12-23 00:00:00+08:00	613.5	614.5	601.5	602.0	15623392	0.0	0.0
2025-12-24 00:00:00+08:00	602.5	602.5	602.5	602.5	0	0.0	0.0
2025-12-29 00:00:00+08:00	606.0	611.0	596.0	596.5	18502650	0.0	0.0
2025-12-30 00:00:00+08:00	598.5	601.0	594.0	600.0	13582535	0.0	0.0
2025-12-31 00:00:00+08:00	599.0	599.0	599.0	599.0	0	0.0	0.0
2026-01-02 00:00:00+08:00	600.5	624.5	600.5	623.0	16200058	0.0	0.0
2026-01-05 00:00:00+08:00	624.0	628.0	615.5	624.5	19947025	0.0	0.0
2026-01-06 00:00:00+08:00	627.0	638.5	626.0	632.5	24168431	0.0	0.0
2026-01-07 00:00:00+08:00	627.5	629.5	615.0	624.5	21378622	0.0	0.0
2026-01-08 00:00:00+08:00	618.5	621.0	610.0	616.0	18742539	0.0	0.0
2026-01-09 00:00:00+08:00	616.0	617.0	610.0	611.0	17813669	0.0	0.0
2026-01-12 00:00:00+08:00	616.0	627.5	613.5	623.0	27530129	0.0	0.0
2026-01-13 00:00:00+08:00	637.0	639.0	622.5	627.5	24212695	0.0	0.0
2026-01-14 00:00:00+08:00	634.0	638.5	626.0	633.0	28677291	0.0	0.0
2026-01-15 00:00:00+08:00	631.0	632.5	618.5	622.0	26194252	0.0	0.0
2026-01-16 00:00:00+08:00	627.0	630.0	613.5	617.5	20616793	0.0	0.0
2026-01-19 00:00:00+08:00	613.5	614.5	608.5	610.0	13233330	0.0	0.0
2026-01-20 00:00:00+08:00	601.0	607.0	598.0	601.0	24332228	0.0	0.0
2026-01-21 00:00:00+08:00	598.0	605.5	598.0	600.0	11476120	0.0	0.0

Extract multiple stocks

- yf.download(tickers, start, end)
- tickers separated by a space

```
import yfinance as yf

ticker = yf.Ticker("0700.HK")

# download Tencent, HSBC, Apple
data = yf.download("0700.HK 0005.HK AAPL", start="2025-01-01", end="2025-06-30")


print(data)
```

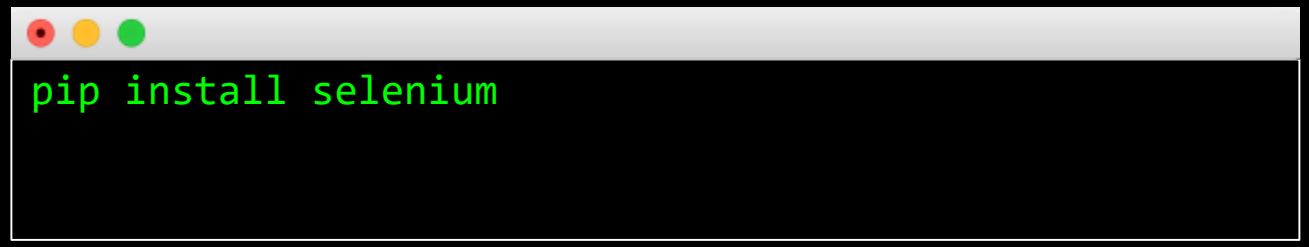
```
Price          Close                                 High  ...        Open      Volume
Ticker      0005.HK     0700.HK        AAPL     0005.HK  ...        AAPL     0005.HK      0700.HK        AAPL
Date                                                     ...
2025-01-02  71.669708  412.400024  242.752106  71.811724  ...  247.809220  15851617.0   20733037.0  55740700.0
2025-01-03  71.006966  410.615601  242.264313  71.717035  ...  242.264313  11755463.0   16843241.0  40244100.0
2025-01-06  71.527695  405.857117  243.896912  71.622369  ...  243.210016  12886461.0   16869318.0  45045600.0
2025-01-07  71.291008  376.315002  241.119492  71.859062  ...  241.886014  18577225.0  142920468.0  40856000.0
2025-01-08  71.906395  366.005035  241.607254  71.906395  ...  240.830767  18019505.0   95456188.0  37628900.0
...               ...         ...         ...        ...  ...         ...         ...          ...         ...
2025-06-23  90.441856  504.000000  201.076660  90.638894  ...  201.206392  11873908.0   14252887.0  55814300.0
2025-06-24  92.707825  509.500000  199.879181  93.101908  ...  202.164363  29990019.0   17768858.0  54064000.0
2025-06-25  94.185631  512.500000  201.136536  95.367882  ...  201.026766  36558811.0   17592461.0  39525700.0
2025-06-26  93.988586  513.000000  200.577698  94.481189  ...  201.006787  10230801.0   15000643.0  50799100.0
2025-06-27  93.594505  513.000000  200.657532  93.840807  ...  201.465827  14778679.0   15181667.0  73188600.0
```

Other attributes

```
import yfinance as yf

msft = yf.Ticker("MSFT")

# get all stock info
msft.info

# get historical market data
hist = msft.history(period="1mo")

# show meta information about the history (requires history() to be called first)
msft.history_metadata

# show actions (dividends, splits, capital gains)
msft.actions
msft.dividends
msft.splits
msft.capital_gains

# only for mutual funds & etfs # show share count
msft.get_shares_full(start="2022-01-01", end=None)

# show financials:
# - income statement
msft.income_stmt
msft.quarterly_income_stmt
# - balance sheet
msft.balance_sheet
msft.quarterly_balance_sheet
# - cash flow statement
msft.cashflow
msft.quarterly_cashflow
# see `Ticker.get_income_stmt()` for more options

# show holders
msft.major_holders
msft.institutional_holders
msft.mutualfund_holders
msft.insider_transactions
msft.insider_purchases
msft.insider_roster_holders

# show recommendations
msft.recommendations
msft.recommendations_summary
msft.upgrades_downgrades

# Show future and historic earnings dates, returns at most next 4 quarters and last 8 quarters by default.
# Note: If more are needed use msft.get_earnings_dates(limit=XX) with increased limit argument.
msft.earnings_dates

# show ISIN code - *experimental*
# ISIN = International Securities Identification Number
msft.isin

# show options expirations
msft.options

# show news
msft.news

# get option chain for specific expiration
opt = msft.option_chain('YYYY-MM-DD')
# data available via: opt.calls, opt.puts
```

Selenium

- Selenium is a web automation tool that allows you to simulate user interactions, perform web scraping, and run automated tests on web applications.
- Official document: https://selenium-python.readthedocs.io/



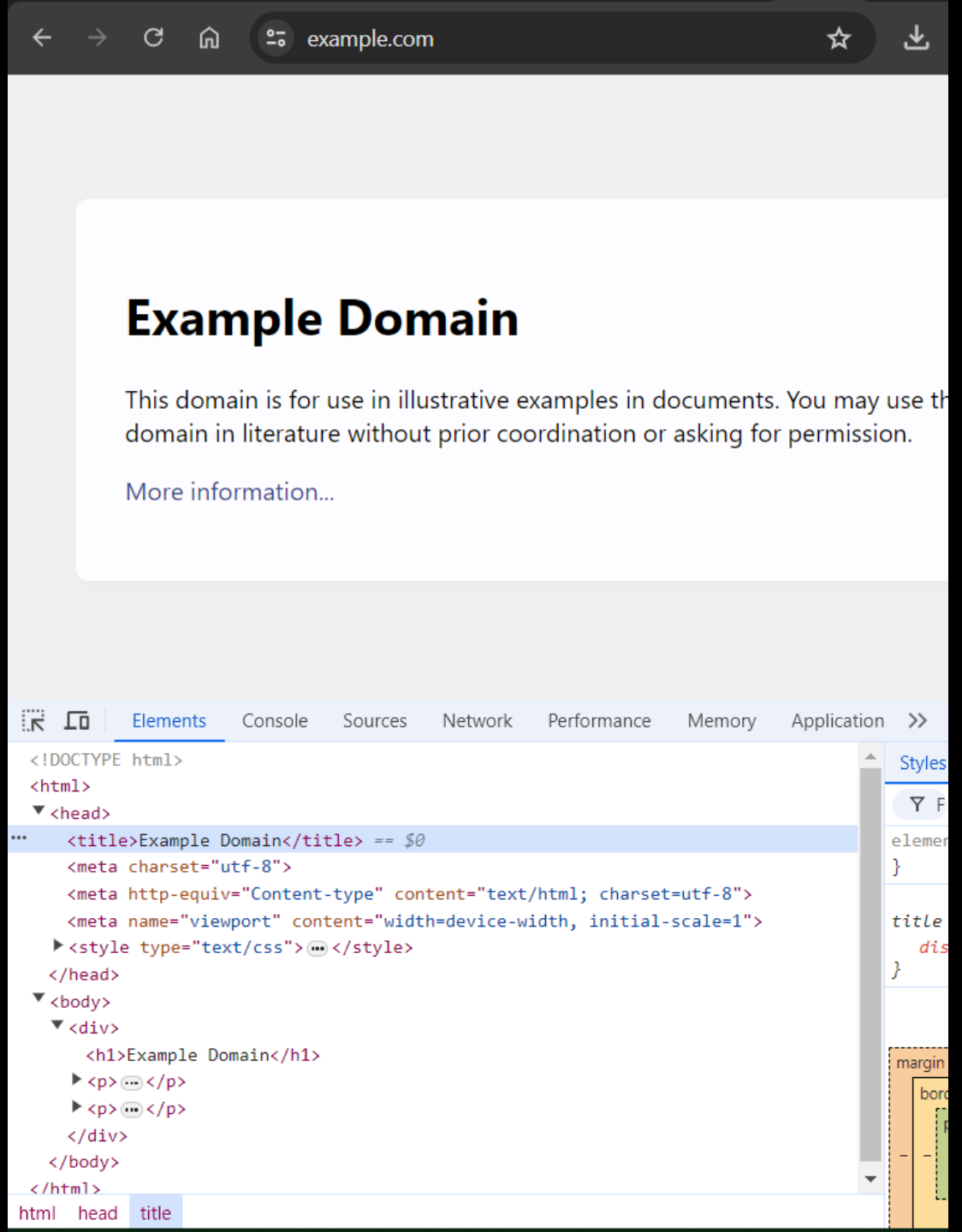
Webdrive download

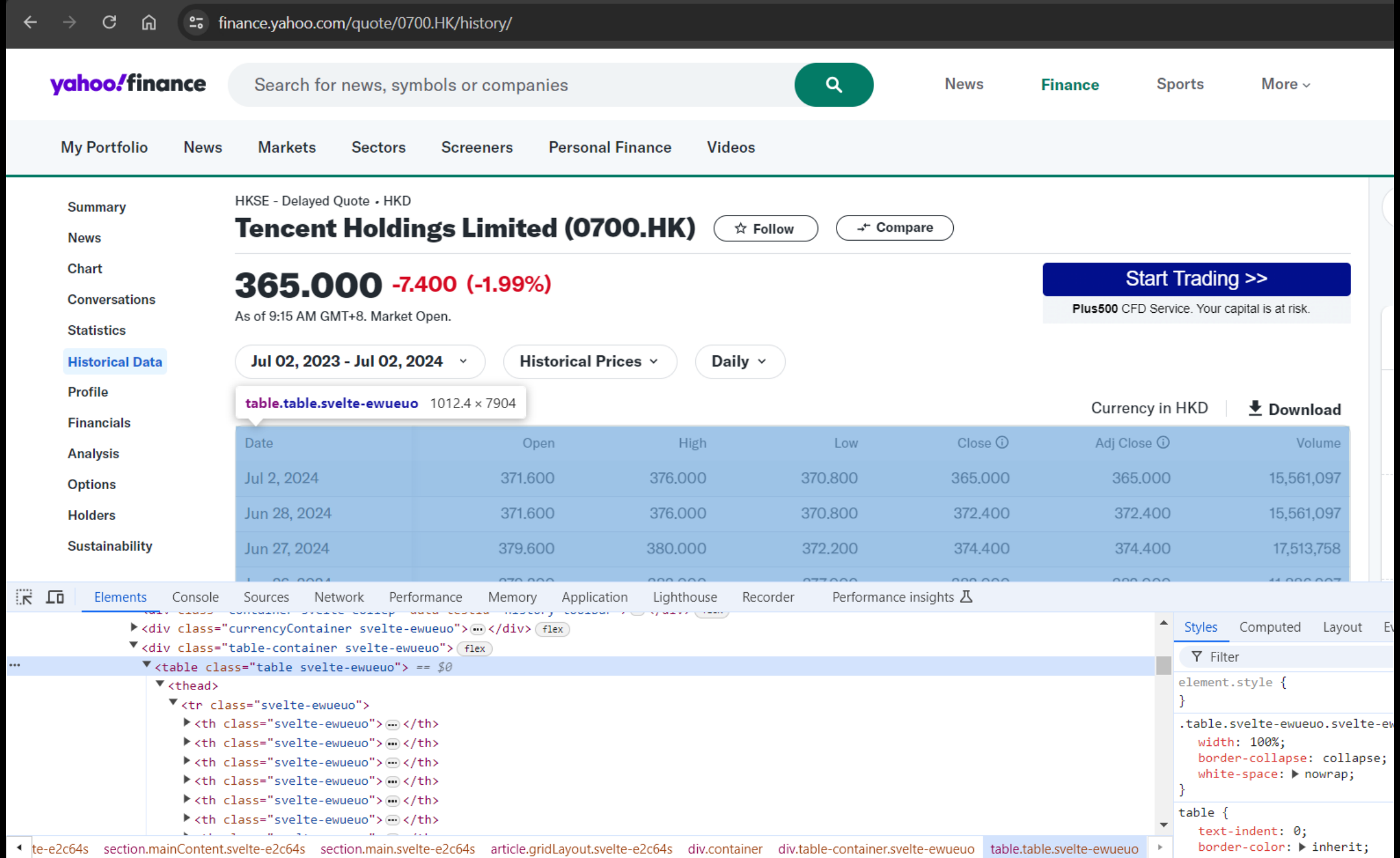
- Chrome: https://googlechromelabs.github.io/chrome-for-testing/#stable
- Edge: https://developer.microsoft.com/en-us/microsoft-edge/tools/webdriver?form=MA13LH
- Firefox: https://github.com/mozilla/geckodriver/releases

Quick Example

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service

chrome_service = Service(executable_path=' path/to/chromedriver')
driver = webdriver.Chrome(service=chrome_service)

driver.get('http://example.com')
print(driver.title)

driver.quit()
```

```
DevTools listening on ws://127.0.0.1:49236/devtools/browser/f14841af-84e0-4681-9cf4-eb2b55f3e68c
Example Domain
```




finance.yahoo.com/quote/0700.HK/history/
yahoo!finance
Search for news, symbols or companies
News
Finance
Sports
More
My Portfolio
News
Markets
Sectors
Screeners
Personal Finance
Videos
Summary
News
Chart
Conversations
Statistics
Historical Data
Profile
Financials
Analysis
Options
Holders
Sustainability
HKSE - Delayed Quote • HKD
Tencent Holdings Limited (0700.HK)
Follow
Compare
365.000 -7.400 (-1.99%)
As of 9:15 AM GMT+8. Market Open.
Start Trading >>
Plus500 CFD Service. Your capital is at risk.
Jul 02, 2023 - Jul 02, 2024
Historical Prices
Daily
table.table.svelte-ewueuo 1012.4 × 7904
Currency in HKD
Download
Date
Open
High
Low
Close
Adj Close
Volume
Jul 2, 2024
371.600
376.000
370.800
365.000
365.000
15,561,097
Jun 28, 2024
371.600
376.000
370.800
372.400
372.400
15,561,097
Jun 27, 2024
379.600
380.000
372.200
374.400
374.400
17,513,758
Elements
Console
Sources
Network
Performance
Memory
Application
Lighthouse
Recorder
Performance insights
… flex
 flex
<table class="table svelte-ewueuo"> == $0
<thead>
<tr class="svelte-ewueuo">
<th class="svelte-ewueuo">…</th>
Styles
Computed
Layout
Filter
element.style {
}
.table.svelte-ewueuo.svelte-ew
width: 100%;
border-collapse: collapse;
white-space: nowrap;
}
table {
text-indent: 0;
border-color: inherit;
te-e2c64s
section.mainContent.svelte-e2c64s
section.main.svelte-e2c64s
article.gridLayout.svelte-e2c64s
div.container
div.table-container.svelte-ewueuo
table.table.svelte-ewueuo

Extract stock price

```
            Date     Open     High      Low    Close Adj. Close      Volume
0    Jan 21, 2026  598.000  605.500  598.000  601.000    601.000  12,012,950
1    Jan 20, 2026  601.000  607.000  598.000  601.000    601.000  24,332,228
2    Jan 19, 2026  613.500  614.500  608.500  610.000    610.000  13,233,330
3    Jan 16, 2026  627.000  630.000  613.500  617.500    617.500  20,616,793
4    Jan 15, 2026  631.000  632.500  618.500  622.000    622.000  26,194,252
..            ...      ...      ...      ...      ...        ...         ...
242  Jan 27, 2025  388.000  398.000  388.000  395.600    392.177  23,605,938
243  Jan 24, 2025  383.200  392.400  382.200  390.600    387.220  24,079,119
244  Jan 23, 2025  383.000  386.000  379.600  381.200    377.901  21,256,810
245  Jan 22, 2025  385.000  388.600  381.000  383.400    380.082  17,859,069
246  Jan 21, 2025  392.000  392.200  386.400  387.400    384.048  25,249,668

[247 rows x 7 columns]
```

```python
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from bs4 import BeautifulSoup
import time
import pandas as pd

# Path to your WebDriver (adjust the path to where you downloaded the WebDriver)
webdriver_path = 'D:/program/chromedriver.exe'

# Set up the WebDriver
service = Service(webdriver_path)
driver = webdriver.Chrome(service=service)

url = 'https://finance.yahoo.com/quote/0700.HK/history/'
driver.get(url)

# Wait for the page to load
WebDriverWait(driver, 10).until(
    EC.presence_of_element_located((By.CSS_SELECTOR, 'table[class="table yf-1m2i7s2 noDl hideOnPrint"]'))
)

# Scroll down to load more data if necessary
# Adjust the range for more scrolling, if required
for _ in range(3):
    driver.find_element(By.TAG_NAME, 'body').send_keys(Keys.END)
    time.sleep(2)

# Wait for the new data to load # Get the page source and parse it with BeautifulSoup
page_source = driver.page_source
soup = BeautifulSoup(page_source, 'html.parser')

# Find the historical data table
table = soup.find('table', {'class': 'table yf-1jecxey noDl hideOnPrint'})
rows = table.find_all('tr')

# Extract and print the data
data = []
for row in rows[1:]: # Skip the header row
    cols = row.find_all('td')
    if len(cols) > 6:
        date = cols[0].text
        open_price = cols[1].text
        high_price = cols[2].text
        low_price = cols[3].text
        close_price = cols[4].text
        adj_close_price = cols[5].text
        volume = cols[6].text
        data.append([date, open_price, high_price, low_price, close_price, adj_close_price, volume])

# crate data frame
df = pd.DataFrame(data, columns=["Date", "Open", "High", "Low", "Close", "Adj. Close", "Volume"])
print(df)

# close driver
driver.quit()
```

Selenium Options

- https://www.selenium.dev/selenium/docs/api/rb/Selenium/WebDriver/Chromium/Options.html#add_argument-instance_method
- headless: open browser in background
- add_emulation: emulate opening the browser in different device

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument('--headless=new')

driver = webdriver.Chrome(CHROMEDRIVER_PATH, options=options)
```

Wrap up

Several Python libraries are introduced for web scraping

- BeautifulSoup
- lxml
- Selenium
- yfinance – specific for Yahoo Finance

Limitation of Data Scraping

1. **Data Quality**:
   - Website data may be incomplete and not up-to-dated
2. **Resource Intensive**:
   - Consume lots of bandwidth and storage when scraping large amount of data
   - Time consuming to fully load a website content to extract the data
3. **Technical Challenges**:
   - Website structure may be changed frequently
   - Many websites may implement anti-bot measures (eg. IP blocking, rate limit, etc)

Database Management

Why use a database for algo-trading?

- Efficient storage and retrieval of large amounts of historical data for strategy backtesting
- Keep data in a local disk for future analysis instead of repeated web scraping
- Data integrity and consistency
- Advanced querying capabilities

SQL Syntax Overview

- **SELECT**: Retrieve data from a table

```
SELECT * FROM users;
```

- **INSERT**: Add new data to a table

```
INSERT INTO users (name, age) VALUES ('Amy', 18);
```

- **UPDATE**: Modify existing data in a table

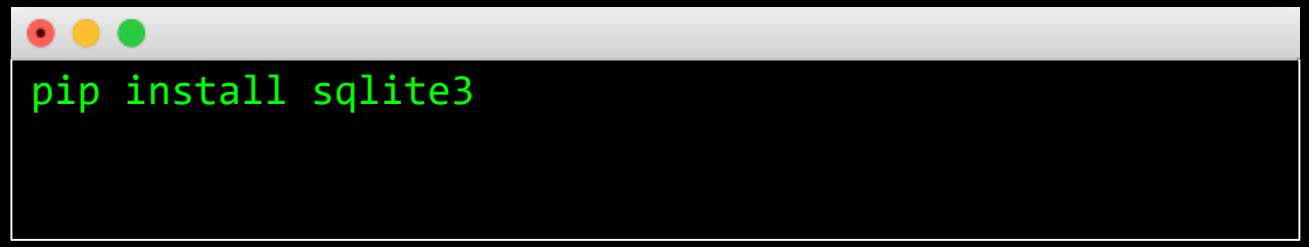
```
UPDATE users SET age=28 WHERE name='Amy';
```

- **DELETE**: Remove data from a table

```
DELETE FROM users WHERE name='Amy';
```

Using SQLite in Python

- SQLite is a C library that provides a lightweight, disk-based database.
- Python has built-in support for SQLite.



Basic Usage

```
import sqlite3

# Connect to a database (or create one if it doesn't exist)
conn = sqlite3.connect('example.db')

# Create a cursor object
cursor = conn.cursor()

# Execute SQL queries using the cursor object
cursor.execute('SELECT * FROM users')

# Fetch all results
rows = cursor.fetchall()

# Close the connection
conn.close()
```

Create a Table

SQL Syntax

```sql
CREATE TABLE users (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER
);
```

Python Example

```python
import sqlite3

conn = sqlite3.connect('example.db')
cursor = conn.cursor()

# Create a table
cursor.execute('''
CREATE TABLE users (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER
)
''')

conn.commit()
conn.close()
```

Inserting Data

SQL Syntax

```
INSERT INTO users (name, age) VALUES ('Alice', 30);
```

Python Example

```
conn = sqlite3.connect('example.db')
cursor = conn.cursor()

# Insert data
cursor.execute('''
INSERT INTO users (name, age) VALUES (?, ?)
''', ('Alice', 30))

conn.commit()
conn.close()
```

Querying Data

SQL Syntax

```
SELECT * FROM users WHERE age > 25;
```

Python Example

```
conn = sqlite3.connect('example.db')
cursor = conn.cursor()

# Query data
cursor.execute('SELECT * FROM users WHERE age > 25')
rows = cursor.fetchall()

for row in rows:
    print(row)

conn.close()
```

Updating Data

SQL Syntax

```
UPDATE users SET age = 31 WHERE name = 'Alice';
```

Python Example

```
conn = sqlite3.connect('example.db')
cursor = conn.cursor()

# Update data
cursor.execute('''
UPDATE users SET age = ? WHERE name = ?
''', (31, 'Alice'))

conn.commit()
conn.close()
```

Deleting Data

SQL Syntax

```
DELETE FROM users WHERE name = 'Alice';
```

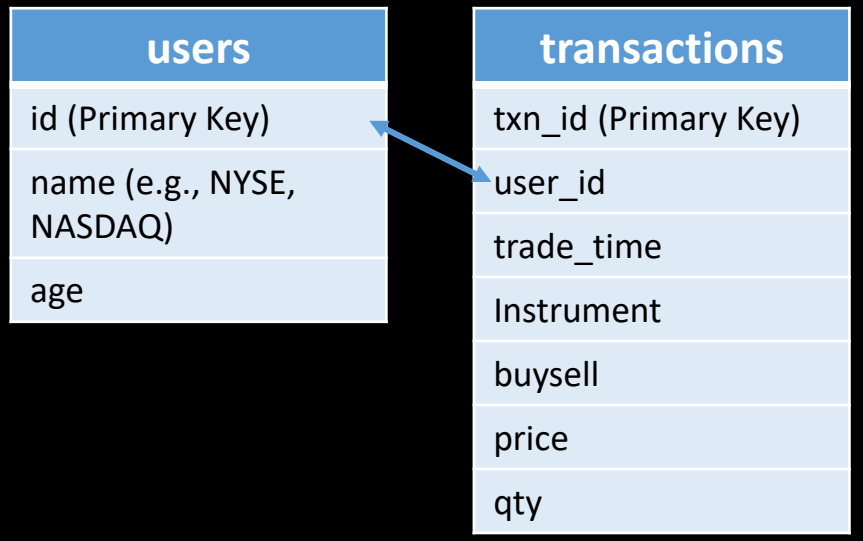
Python Example

```
conn = sqlite3.connect('example.db')
cursor = conn.cursor()

# Delete data
cursor.execute('''
DELETE FROM users WHERE name = ?
''', ('Alice',))

conn.commit()
conn.close()
```

Querying Data with table join

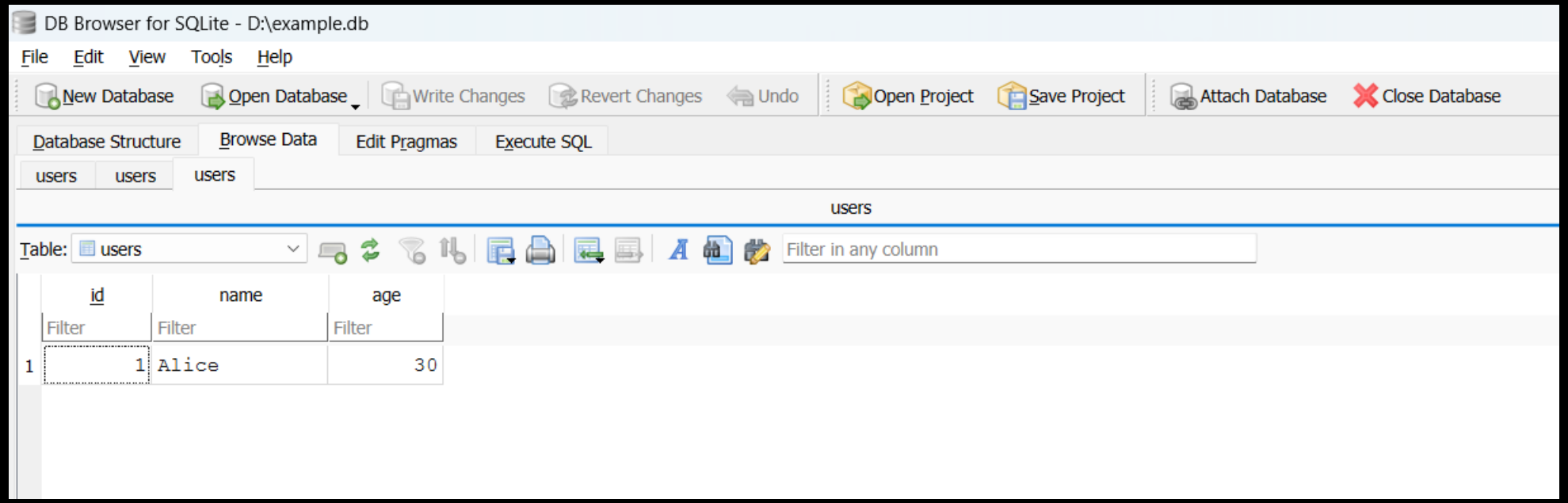
Suppose we want to know the total transactions done by each client.

```
SELECT A.id, A.name, count(B.*)
FROM users A
INNER JOIN transactions B ON A.id = B.user_id
GROUP BY A.id, A.name;
```



SQLite Browser

- SQLite Browser (https://sqlitebrowser.org/) is an open sourced tool to create, edit, search and visualize SQLite database files.



Scrape Market Data into database

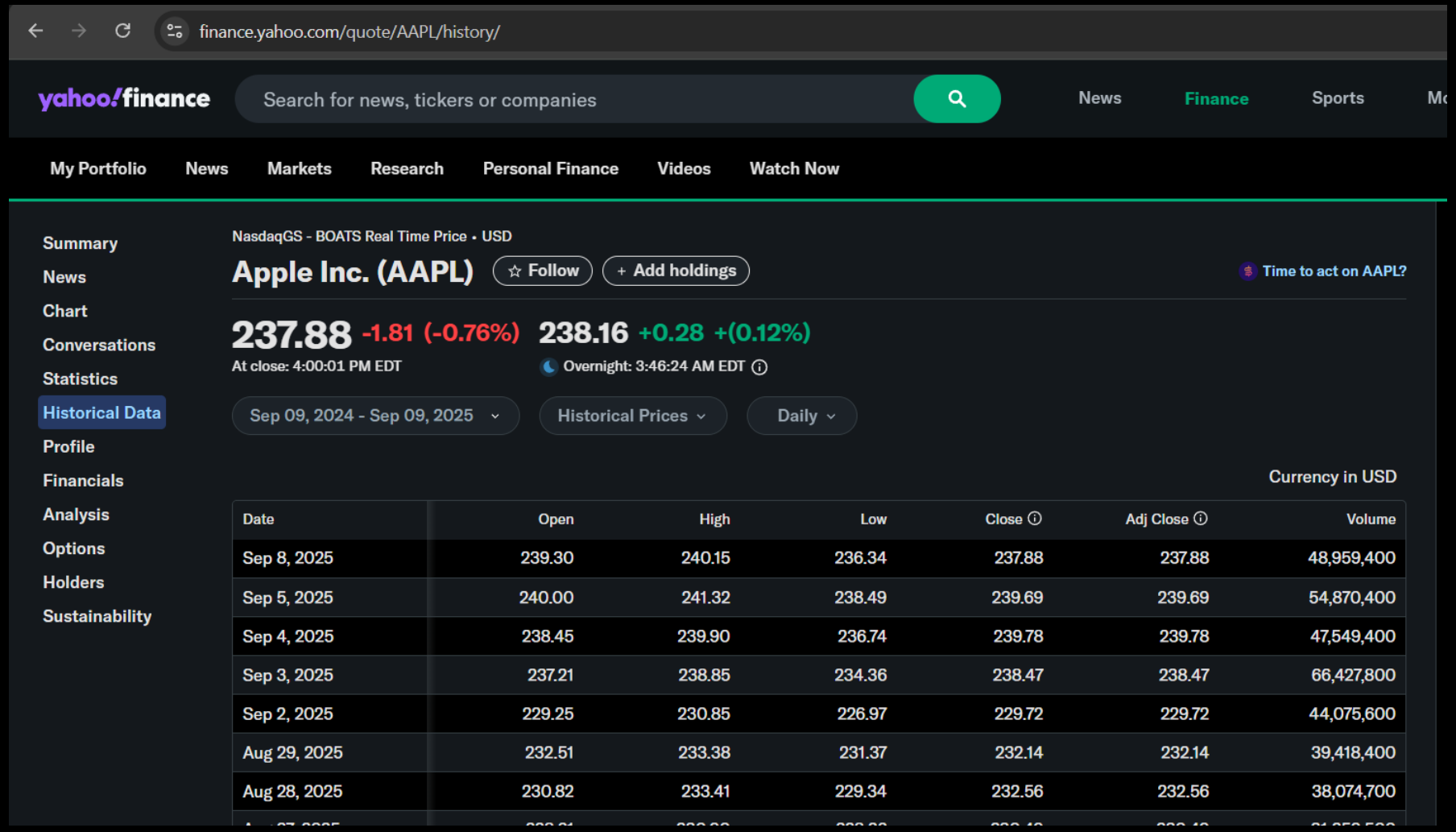
```
import sqlite3
import yfinance as yf

conn = sqlite3.connect('D:/example.db')
cursor = conn.cursor()

# create table
cursor.execute('''
CREATE TABLE market_candles (
    symbol text,
    timestamp text,
    open_price float,
    high_price float,
    low_price float,
    close_price float,
    volume float
)''')

# download data from yahoo finance
symbol = "0700.HK"

ticker = yf.Ticker(symbol)
data = ticker.history(period='max')

# save to database
for index, row in data.iterrows():
    t = index
    o = row['Open']
    h = row['High']
    l = row['Low']
    c = row['Close']
    v = row['Volume']
    sql = """INSERT INTO market_candles VALUES ('{symbol}','{t}',{o},{h},{l},{c},{v})""".format(symbol=symbol, t=t, o=o, h=h, l=l, c=c, v=v)
    cursor.execute(sql)

# close database
conn.commit()
conn.close()
```

Querying market data from database

```
('0700.HK', '2025-06-02 00:00:00+08:00', 493.0, 499.20001220703125, 489.6000061035156, 498.3999938964844,
13086586.0)
('0700.HK', '2025-06-03 00:00:00+08:00', 504.0, 505.5, 501.0, 505.0, 14609189.0)
('0700.HK', '2025-06-04 00:00:00+08:00', 510.0, 513.0, 507.0, 512.0, 18378499.0)
('0700.HK', '2025-06-05 00:00:00+08:00', 517.5, 517.5, 509.0, 515.0, 19329282.0)
('0700.HK', '2025-06-06 00:00:00+08:00', 515.5, 516.5, 511.0, 515.0, 13137101.0)
('0700.HK', '2025-06-09 00:00:00+08:00', 520.0, 521.0, 512.5, 518.0, 18481403.0)
('0700.HK', '2025-06-10 00:00:00+08:00', 519.5, 520.0, 508.5, 513.5, 17039003.0)
('0700.HK', '2025-06-11 00:00:00+08:00', 517.0, 518.0, 514.5, 518.0, 14784979.0)
('0700.HK', '2025-06-12 00:00:00+08:00', 518.0, 518.0, 508.0, 510.0, 13368048.0)
('0700.HK', '2025-06-13 00:00:00+08:00', 510.5, 515.5, 506.0, 510.0, 19085310.0)
('0700.HK', '2025-06-16 00:00:00+08:00', 507.0, 512.0, 504.5, 509.5, 13784320.0)
('0700.HK', '2025-06-17 00:00:00+08:00', 514.0, 514.0, 506.5, 513.5, 11524031.0)
('0700.HK', '2025-06-18 00:00:00+08:00', 510.0, 511.0, 503.5, 508.0, 15187515.0)
('0700.HK', '2025-06-19 00:00:00+08:00', 503.0, 506.0, 496.0, 498.0, 19387767.0)
('0700.HK', '2025-06-20 00:00:00+08:00', 504.5, 505.5, 496.0, 505.5, 20622275.0)
('0700.HK', '2025-06-23 00:00:00+08:00', 498.20001220703125, 504.0, 494.6000061035156, 504.0, 14252887.0)
('0700.HK', '2025-06-24 00:00:00+08:00', 508.0, 511.0, 504.0, 509.5, 17768858.0)
('0700.HK', '2025-06-25 00:00:00+08:00', 512.0, 515.0, 508.0, 512.5, 17592461.0)
('0700.HK', '2025-06-26 00:00:00+08:00', 512.0, 513.0, 507.5, 513.0, 15000643.0)
('0700.HK', '2025-06-27 00:00:00+08:00', 512.0, 514.5, 510.0, 513.0, 15181667.0)
```

```python
import sqlite3

conn = sqlite3.connect('D:/example.db')
cursor = conn.cursor()

# query table
sql = """SELECT *
FROM market_candles
WHERE symbol='0700.HK'
    AND timestamp>='2025-06-01'
    AND timestamp<='2025-06-30'
ORDER BY timestamp;"""

for row in cursor.execute(sql):
    print(row)

# close database
conn.close()
```

Close vs Adjusted Close



Date	Open	High	Low	Close	Adj Close	Volume
Sep 8, 2025	239.30	240.15	236.34	237.88	237.88	48,959,400
Sep 5, 2025	240.00	241.32	238.49	239.69	239.69	54,870,400
Sep 4, 2025	238.45	239.90	236.74	239.78	239.78	47,549,400
Sep 3, 2025	237.21	238.85	234.36	238.47	238.47	66,427,800
Sep 2, 2025	229.25	230.85	226.97	229.72	229.72	44,075,600
Aug 29, 2025	232.51	233.38	231.37	232.14	232.14	39,418,400
Aug 28, 2025	230.82	233.41	229.34	232.56	232.56	38,074,700

https://finance.yahoo.com/quote/AAPL/history/

Close vs Adjusted Close

- The original price series may show sudden jumps or drops due to corporate events (e.g. dividends, stock splits, reverse splits).
- After a dividend payment, the stock price typically drops by $S' = S - D$
- Following a stock split (e.g. 2:1 split), the stock price decreases by a factor of the split ratio (i.e. $S' = S/2$)
- Adjusted close prices are backward updated to account for these corporate events
  - so the values of the adjusted close series may change from time to time

How Yahoo Finance adjust data?

Multipliers

Split multipliers are determined by the split ratio.

For example:

- In a 2 for 1 split, the pre-split data is multiplied by 0.5.
- In a 4 for 1 split, the pre-split data is multiplied by 0.25.
- In a 1 for 5 reverse split, the pre-split data is multiplied by 5.

Dividend multipliers are calculated based on dividend as a percentage of the price, primarily to avoid negative historical pricing.

For example:

- If a $0.08 cash dividend is distributed on Feb 19 (ex- date), and the Feb 18 closing price is $24.96, the pre-dividend data is multiplied by (1-0.08/24.96) = 0.9968.
- If a $2.40 cash dividend is distributed on May 12 (ex- date), and the May 11 closing price is $16.51, the pre-dividend data is multiplied by (1-2.40/16.51) = 0.8546.
- If a $1.25 cash dividend is distributed on Jan 25 (ex- date), and the Jan 24 closing price is $51.20, the pre-dividend data is multiplied by (1-1.25/51.20) = 0.9756.

Reference: https://help.yahoo.com/kb/SLN28256.html

Here's how split and dividend multipliers are calculated and applied to determine adjusted close prices.

The first table shows historical prices and the dates of a split and a dividend. The second table shows the calculations.

The multipliers we'll use are derived from the split and dividend:

- Split Multiplier = 0.5
- Dividend Multiplier = 1 - (0.08/24.96) = 0.9968

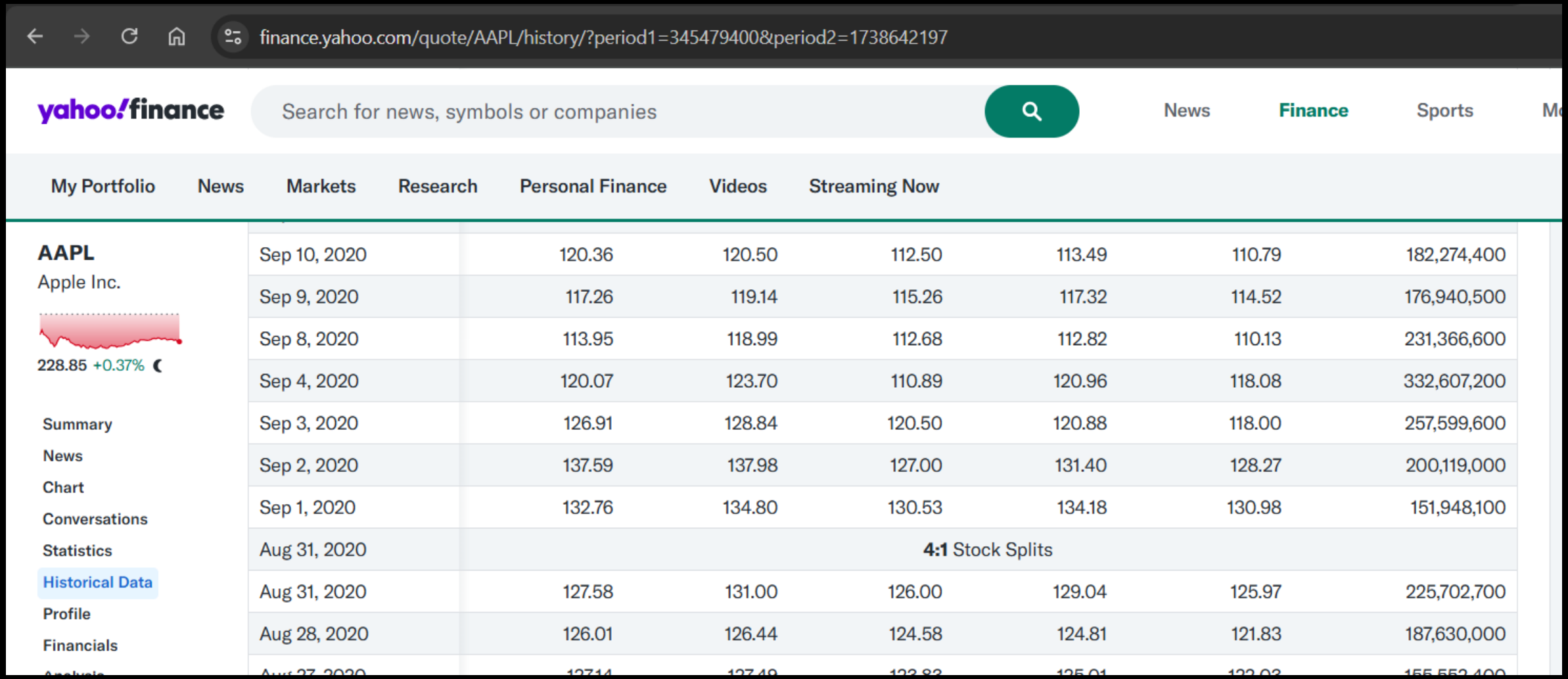
Using these split and dividend multipliers, adjusted close prices are calculated for dates prior to the split and prior to the dividend ex-date.

- The close price for 2/16 and 2/17 are adjusted for both the split on 2/18 and the ex-dividend date, 2/21.
- The close price for 2/18 through 2/20 are adjusted for the ex-dividend date, 2/21.
- The close price for 2/18 through 2/20 don't need adjustment for the split that occurred before close on 2/18.
- The close price for 2/21 and 2/22 aren't adjusted because they're after the split and ex-dividend dates.

Date	Close, Dividend, or Split
2/16	Close = 46.99
2/17	Close = 48.30
2/18	Split = 2:1 Close = 24.96
2/19	Close = 24.91
2/20	Close = 24.95
2/21	Dividend = 0.08 (ex-date) Close = 24.53
2/22	Close = 24.54

Date	Adjusted close calculation
2/16	0.5 * 0.9968 * 46.99 = 23.42
2/17	0.5 * 0.9968 * 48.30 = 24.07
2/18	0.9968 * 24.96 = 24.88
2/19	0.9968 * 24.91 = 24.83
2/20	0.9968 * 24.95 = 24.87
2/21	24.53
2/22	24.54

Any adjustments on OHL data?



Sep 10, 2020	120.36	120.50	112.50	113.49	110.79	182,274,400
Sep 9, 2020	117.26	119.14	115.26	117.32	114.52	176,940,500
Sep 8, 2020	113.95	118.99	112.68	112.82	110.13	231,366,600
Sep 4, 2020	120.07	123.70	110.89	120.96	118.08	332,607,200
Sep 3, 2020	126.91	128.84	120.50	120.88	118.00	257,599,600
Sep 2, 2020	137.59	137.98	127.00	131.40	128.27	200,119,000
Sep 1, 2020	132.76	134.80	130.53	134.18	130.98	151,948,100
Aug 31, 2020			**4:1** Stock Splits			
Aug 31, 2020	127.58	131.00	126.00	129.04	125.97	225,702,700
Aug 28, 2020	126.01	126.44	124.58	124.81	121.83	187,630,000

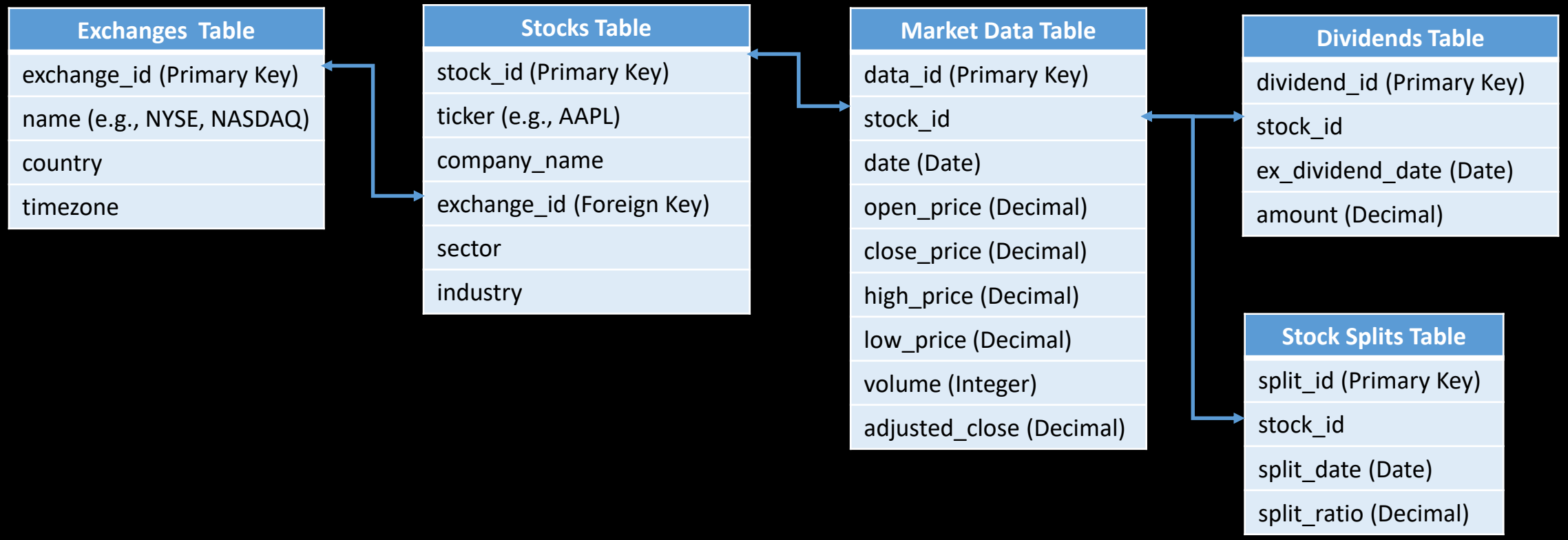
Should we keep the original or the adjusted price in database?

- Pros:
  - Adjusted close provides a more accurate reflection of a stock's value over time
  - Adjusted close is ideal for long-term analysis and backtesting strategies, as it smooths out price jumps from events
  - Ensure consistency across datasets, especially when comparing stocks with varying corporate events.
- Cons:
  - Storing both may require more space
  - Need to regularly update the whole adjusted close series for new events.
  - Adjusted close is not the actual price observed in the market. Modelling on adjusted close could be inaccurate.

Database Design

- Suppose you are going to build a database to store all market data history from global stocks exchanges. How would you structure the database?

Database Design (1)

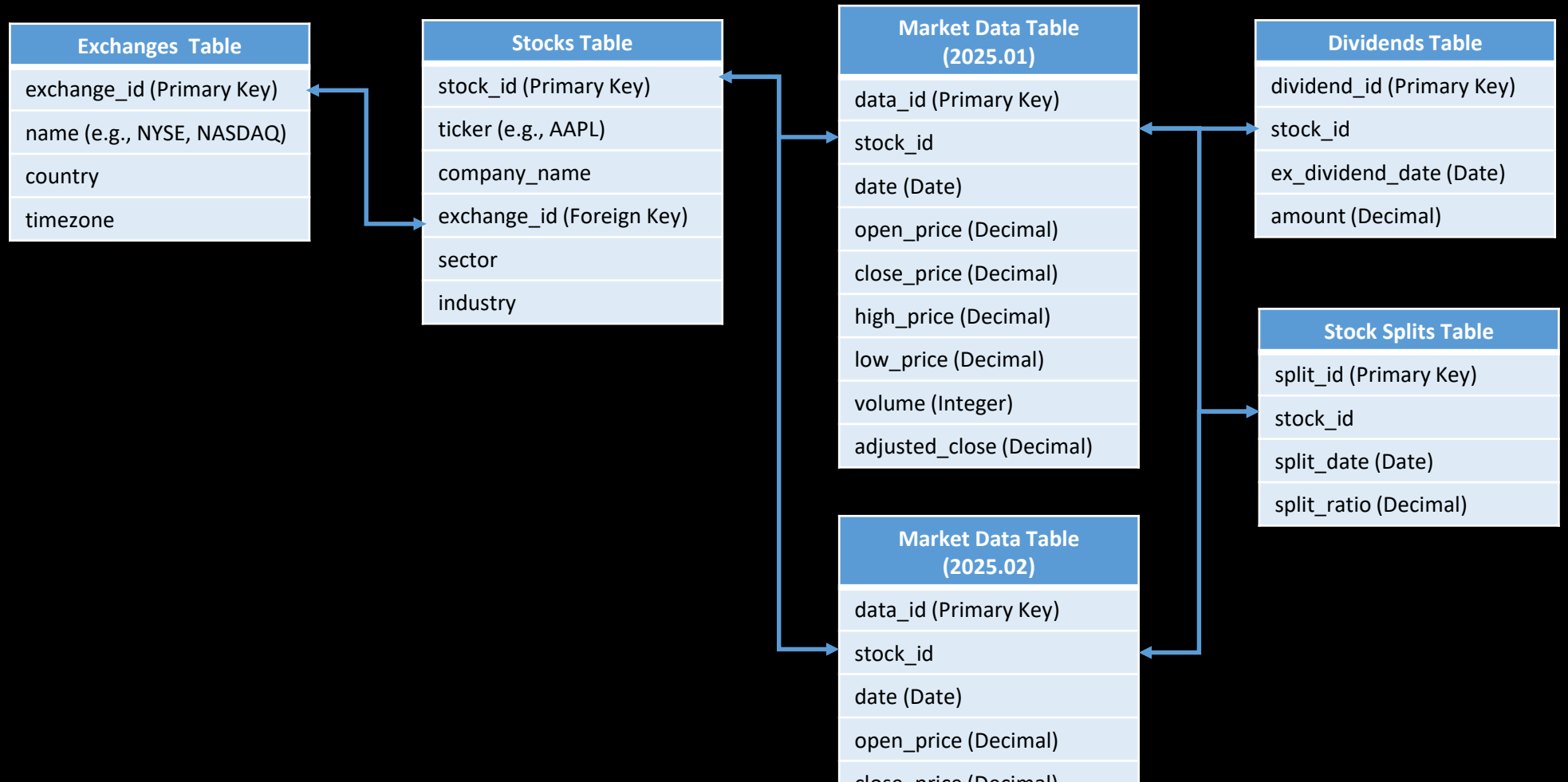


Potential issues for design 1

- The table size of “Market Data Table” will be very large as all stock data is put under the same table
- Data backup and data query speed from “Market Data Table” could be very slow

Database Design (2)

Split the Market Data Table into monthly or quarterly partitions based on the date field.

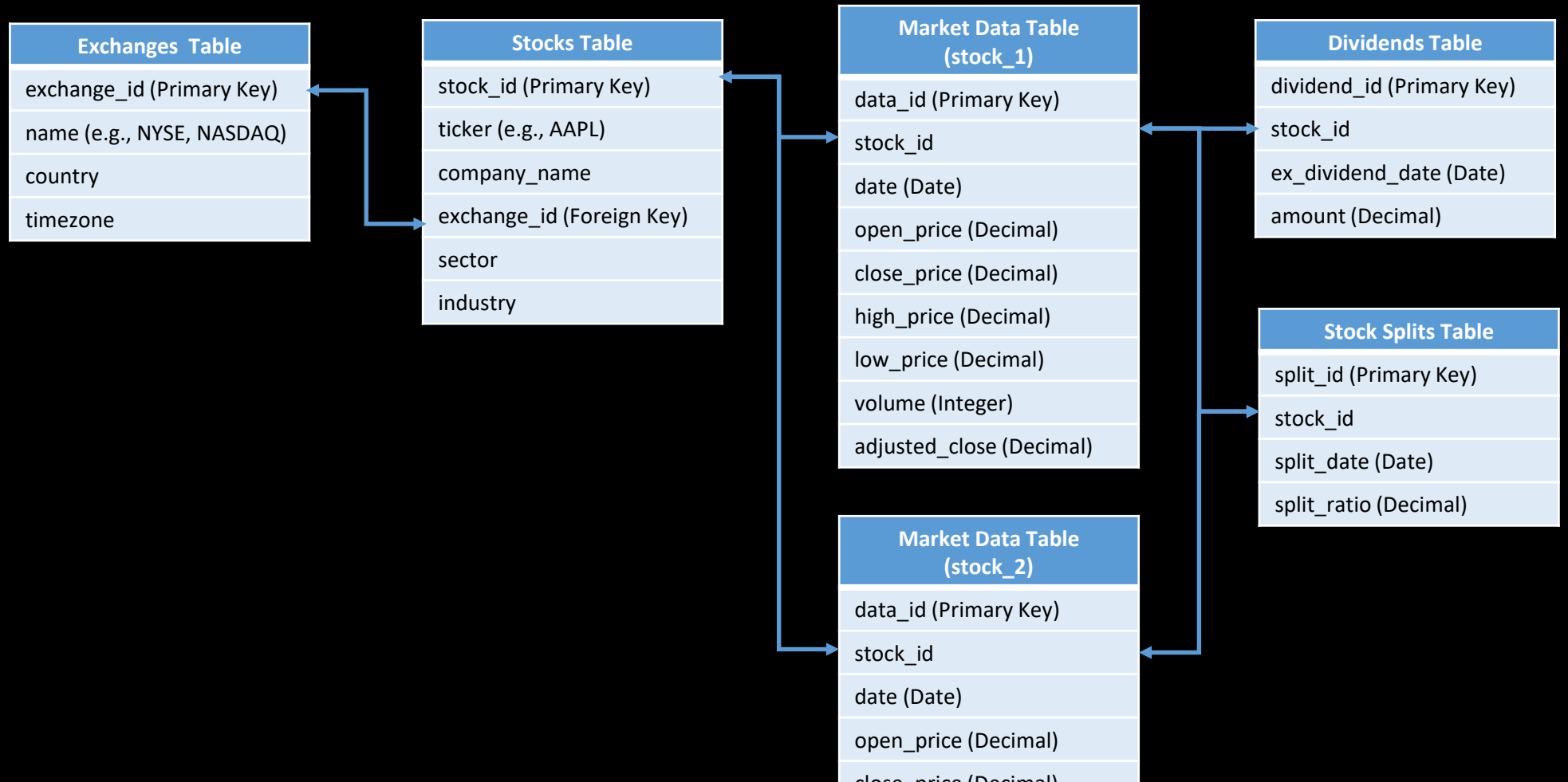


Potential issues for design 2

- Difficult to conduct data aggregation/summary for cross month analysis
- Need a function to generate dynamic SQL statements due to different market data table name

Database Design (3)

Split the Market Data Table into partitions based on the stock_id field.



Potential issues for design 3

- Difficult for cross assets analysis
- Need a function to generate dynamic SQL statements due to different market data table name

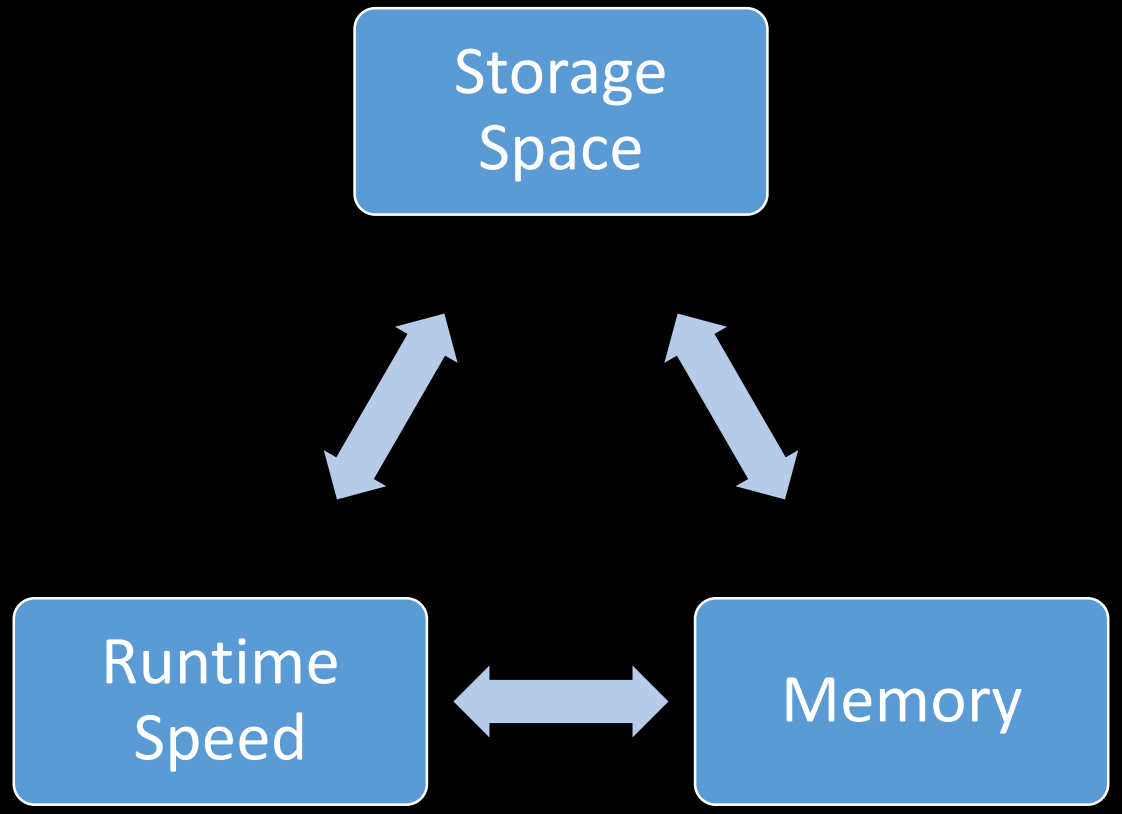
What would be the best design?

Database Design

- There is no single design that is perfect for all situations!
- It depends on our common use cases.
  - For backtest purpose, partition by instruments could be a good choice
  - For data analysis purpose, partition by date could be a better design

Database Design

- Always need to strike a balance between



Key Takeaways

- Learn about the basic structure of a website (i.e. HTML, CSS, Javascript)
- Apply various python libraries for web scraping
  - BeautifulSoup
  - lxml
  - Selenium
  - yfinance
- Understand the limitations of web scraping
- Learn how to create and manage a SQLite database using python
- Considerations for design a database for financial market data